

分 类 号 _____

学号 M201672683

学校代码 10487

密级 _____

华中科技大学

硕士学位论文

基于机器学习的固态缓存优化研究

学位申请人：易铎波

学 科 专 业：计算机系统结构

指 导 教 师：王 桦 副教授

答 辩 日 期：2019 年 5 月 29 日

**A Thesis Submitted in Partial Fulfillment of the Requirements
for the Degree of Master of Engineering**

**Research on SSD Caching Optimization Based
on Machine Learning**

Candidate : Yi Xinbo

Major : Computer Architecture

Supervisor : Associate Prof. Wang Hua

Huazhong University of Science & Technology

Wuhan, Hubei 430074, P.R.China

May 29, 2019

摘要

近年来, SSD 凭借其较高的读写性能及性价比, 在缓存系统中发挥越来越重要的作用。通常缓存空间远小于后端存储空间, 容量差距大约在一个数量级甚至更多, 因此 SSD 缓存的写入密度(单位时间和空间的写入量)远远高于 HDD 后端存储的写入密度, 这给保障 SSD 的寿命带来了巨大挑战。同时, 在社交网络的工作负载下, 对 SSD 缓存的大量写入是不必要的。例如, 在某知名公司的社交图片系统中, 大约 61% 的图片只会被访问一次, 但它们仍然被写入了缓存。由于这些图片只访问一次, 进入缓存后不会被再次访问, 对缓存而言写入这样的图片就是无用的。因此, 如果能够主动地预测这类图片并阻止它们进入缓存, 将消除对 SSD 缓存的不必要写入, 提高缓存空间利用率, 并延长 SSD 缓存的使用寿命。

为了实现这一目标, 提出了一个适用于不同缓存空间大小的一次访问判定标准, 进一步提出了一次访问过滤策略。在此基础上, 设计了一个用于预测的分类器来实现这一策略。与目前为止最先进的基于历史信息的预测方案不同, 预测需要在文件第一次访问时就做出判定, 也就无法借助文件的历史访问信息来完成, 这对于实现良好的预测准确性带来了极大挑战。为了解决这个问题, 将决策树集成到分类系统中, 提取用户的社交信息作为分类特征, 并综合使用代价敏感学习、历史表、训练模型定时更新等方法来提高分类精度。基于该技术方案, 预测准确度达到了 80% 以上, 大幅度地降低了 SSD 写入次数, 提高了缓存命中率。实验结果表明: 一次访问过滤策略使缓存多个方面的性能得到了提升, 以 LRU 为例, 使用一次访问过滤策略后缓存命中率提高了 4.4%, 对 SSD 的写入率降低了 56.8%, 平均访问延迟降低了减少 5.5%。

关键词: 固态缓存 图片缓存 机器学习 社交网络

Abstract

SSD has been playing a significantly important role in caching systems due to high read-write performance and cost-effective. Since cache space is much smaller than that of the backend storage by one order of magnitude or even more, write density (writes per unit time and space) of SSD cache is therefore much higher than that of HDD storage, which brings about great challenges to SSD's lifetime. Meanwhile, under social network workloads, quite a lot writes on SSD are unnecessary, e.g., in a famous company's photo caching, there is 61% of total photos are just accessed once whereas they are still swapped in and out of the cache. Since these images are only accessed once, they will not be accessed again after entering the cache. It is useless to write such images to the cache. Therefore, if we can predict this kind of photos proactively and prevent them from entering the cache, we can eliminate unnecessary SSD cache writes and improve cache space utilization.

To cope with the challenge, we put forward a "one-time-access criteria" that is applied to cache space, and further propose a "one-time-access-exclusion" policy. Based on that, we design a prediction-based classifier to facilitate the policy. Unlike the state-of-the-art history-based predictions, our prediction is non-history-oriented, the prediction here needs to be judged on the first access, and there is no history information, which is challenging to achieve a good prediction accuracy. To address this issue, we integrate a decision tree into the classifier, extract social-related information as classifying features, and apply history table, cost-sensitive learning and daily update model to improve classification precision. Due to these techniques, we attain prediction accuracy of over 80%. Experimental results show that "one-time-access-exclusion" approach makes caching performance outstanding in most aspects, taking LRU for instance, hit rate is improved by 4.4%, cache writes are decreased by 56.8%, and the average access latency is dropped by 5.5%.

Keywords: SSD Photo Caching Machine Learning Social Network

目录

摘 要.....	I
Abstract	II
1 绪 论	
1.1 课题研究背景.....	(1)
1.2 国内外研究现状分析.....	(2)
1.3 研究内容.....	(6)
1.4 本章小结.....	(7)
2 相关技术介绍	
2.1 缓存技术.....	(8)
2.2 基于闪存的固态硬盘技术.....	(11)
2.3 机器学习技术.....	(13)
2.4 本章小结.....	(15)
3 一次访问过滤的缓存系统	
3.1 图片缓存系统.....	(16)
3.2 策略核心思想.....	(18)
3.3 缓存架构设计.....	(19)
3.4 一次访问判断标准.....	(21)
3.5 本章小结	(24)
4 一次访问分类预测	
4.1 数据采样.....	(25)
4.2 分类算法.....	(26)

华中科技大学硕士学位论文

4.3 特征提取.....	(28)
4.4 分类预测质量的保障机制.....	(30)
4.5 本章小结.....	(33)
5 系统测试与分析	
5.1 测试环境.....	(34)
5.2 数据来源和采样.....	(34)
5.3 分类系统的性能.....	(35)
5.4 缓存系统的性能.....	(35)
5.5 分类系统的分解.....	(52)
5.6 本章小结.....	(53)
6 全文总结与展望.....	(54)
致 谢.....	(55)
参考文献.....	(56)
附录 硕士研究生期间的研究成果.....	(63)

1 绪论

1.1 课题研究背景

长期以来, HDD 一直是信息存储的主要存储介质。进入大数据时代后, 各种应用服务对存储都有了更高的要求, 存储系统面临着更严峻的性能挑战。Facebook, 腾讯等商业公司开始尝试使用基于闪存的 SSD 来提升服务的性能, 构建一个基于 HDD 存储层之上的缓存系统。目前, 对 SSD 缓存系统的研究主要针对两个目标: 延长 SSD 的使用寿命和提升 SSD 缓存系统的性能。

(1) 延长 SSD 的使用寿命

影响 SSD 寿命的一个关键因素是对 SSD 的写入量, 通常称之为“写入流量”。不幸的是, 与把 SSD 作为普通的后端存储相比, 对 SSD 缓存的写入流量比后端存储系统明显要多得多。原因在于缓存系统的基本目标就是用一个较小的高速存储空间保存流行内容, 吸收对这部分内容的访问请求, 减少对后端存储系统压力。以典型的缓存结构为例: 用一个大小为 1TB 的 SSD 作缓存层, 服务于由 10 个 HDD ($10 * 2TB$) 组成的后端存储磁盘阵列。假设对各存储单元的访问是均匀分布的, SSD 缓存层与后端存储系统的写入密度(定义为单位时间和空间内的写入次数)的比例大约是 20: 1。可见, 当 SSD 被用作缓存介质而不是后端存储时, 将接受到大量密集地写入操作。由于 SSD 的写次数直接决定了其寿命, 因此, 如何有效地管理写入操作、减少写入次数与写入流量成为 SSD 缓存研究的关键问题。

(2) 提升 SSD 缓存系统的性能

SSD 缓存的性能提升主要通过两种方式实现: 提升硬件和改进缓存算法。理想情况下, 我们希望有足够大的 SSD 空间来容纳所有的数据。而实际上, SSD 缓存空间通常比后端存储系统的空间小得多, 无法容纳所有的数据。另一方面, 选择更好的算法可以较大地提升缓存性能, 从基本的 LRU, FIFO, LFU 到更高级的 S3LRU^[1], LIRS^[2], ARC^[3]等等。每个算法利用了某种访问局部性, 例如时间局部性、空间局部性和内容局部性, 因此适用于不同的场景, 没有适用于所用场合的通用算法。

中国某知名社交网络服务提供商, 其即时交流软件, 如微信和 QQ 等。它们为数十

亿用户提供服务，拥有庞大的用户数据量，并且更重要的是这些用户数据是以很快的速度在增长。以 QQ 为例，到 2017 年 8 月，有 2 万亿张用户图片，总计约 300 PB 数据，并且每天都会新增加 3 亿张用户图片，每天用户的浏览次数达 500 亿次。为了应对如此大规模的密集访问，在缓存服务器上配备了 SSD 以提升性能。在过去的两年来，我们一直在与该公司的工程师团队合作，共同优化其图片缓存系统。最初，尝试了扩展缓存空间和采用不同的缓存算法，但收效甚微。原因之一是当前的缓存空间已经非常大了。通过进一步分析图片缓存的访问日志，发现大量图片只被访问一次，但它们仍然像其他图片一样被写入到缓存中。这一观察，激发了本文的研究灵感：只访问一次的图片进入缓存后不会被再次访问，对缓存而言写入这样的图片就是无用的。如果可以避免这些图片进入 SSD 缓存，便可以大大减少对 SSD 无效写入，延长 SSD 的寿命。同时，缓存的性能也将因为缓存空间利用率的提升而增加。

1.2 国内外研究现状分析

本研究的目标是避免只访问一次的图片进 SSD 缓存系统，以达到减少对 SSD 的无效写入，增大缓存空间利用率，提升缓存系统性能的目的。研究将借助机器学习方法来完成图片是否只访问一次的判定。在本节中，将主要讨论以下三个方面的相关研究工作。一是缓存替换算法，介绍常见的缓存替换策略以及运用机器学习方法对其进行优化的相关研究；二是固态缓存系统，重点说明使用了固态盘的缓存系统将要面临的问题；三是对图片的热度进行预测，以确定什么样的图片应该进入缓存。

1.2.1 缓存替换算法

多年来，缓存替换算法一直是一个活跃的研究课题。传统的缓存替换算法假设了具有某种访问特性的数据更有可能被再次访问。通常会用到以下几种访问特性，一是时间局部性，即是假设最近访问的文件，更有可能被访问。二是访问频率，即是假设访问频率越高的文件越有可能被再次访问。三是访问顺序，即是最近到来的文件越有可能被再次访问。这三种假设，分别对应了 LRU、LFU、FIFO 三种缓存替换算法。

Least recently used (LRU)：最为经典的缓存替换算法，在每次需要淘汰数据时选择一个最久未被使用的块。通常使用一个链表来实现，当一个块命中时，移动到链表的

头部，因此链表的头部就是最近被使用过的数据，而链表的尾部就是最久未被使用的数据。

Least-frequently used (LFU): 统计每一个缓存数据块的使用频率，每次淘汰时使用最小访问频率的数据块。其实现方式类似于 LRU，链表的头部保留访问频率最高的频率块，尾部保留频率最低的块。

First in first out(FIFO): 它的工作方式类似于一个先入先出的队列，每次访问一个新的数据时放在队列的头部，每次淘汰时从队列的尾部选择一个最早到达的数据。

在这些替换算法之上，还有一些更高级的缓存替换算法，它们试图结合多种访问特性，例如 SLRU^[1]，ARC^[3]。基于这些传统算法，人们对如何在某些特定的场景进行进一步的优化缓存性能做了大量的研究^[4]。Asaf Cidon^[5]和 Qiang Yang^[6]等人讨论如何改进在 Web 环境中的缓存性能。Zaoxing Liu^[7]等人创新性的使用两层缓存系统来实现分布式缓存中的负载均衡。Leeor Peled 等人^[8]运用强化学习的方法，预取语义相关的数据至缓存，提升缓存性能。Georgios Keramidas^[9]和 Mazen Kharbutli^[10]专注于预测数据何时会被重用，从而优化 CPU 的片上缓存性能。Daniel A Jiménez^[11,12] 等人利用机器学习方法达到类似的目的。他的主要思路是模仿 Beladaya 算法，即估计数据的重用距离，借助这一预测结果，设计更好的淘汰策略以达到更高的缓存命中率。而本文的思路是，过滤掉对缓存的无效数据写入，以提升缓存空间的利用率，延长 SSD 寿命。此外，不同于 CPU 的片上缓存，在本文的研究场景中预测重用距离十分困难，因为数据局部性很弱，且数据特征属性与其重用距离之间的相关性较弱。Daniel A Jiménez^[11]，Mazen Kharbutli^[10]，和 Moinuddin K. Qureshi^[13]与本文提出了相似缓存的旁路策略（Cache bypass）以提高缓存性能，但他们都没有提出用于确定哪些数据不应该放入缓存的一个确切标准。

在最近的图片缓存系统研究中，Doug Beaver^[14]等人在 2010 年的 OSDI 大会上介绍的 Facebook 的图片后端存储系统——Haystack。接着，Qi Huang 等人^[15]详细分析了 Facebook 图片缓存服务栈。Subramanian Muralidhar^[16] 等人进一步介绍了 Facebook 的温数据存储系统。Xiao Bai 等人^[17]讨论了图片缓存系统中，如何提升处理不同分辨率图片时的效率。Si Sun 等人^[18] 详细分析了 QQ 相册的访问模式，与本文不同的是，Si

Sun 主要使用缓存预取策略来提升缓存性能,而本文研究缓存的数据准入原则。以上这些研究工作展示了图片缓存的一个基本架构和工作特点。

1.2.2 固态缓存系统

固态盘的读写性能与成本均介于 RAM 和磁盘之间。在磁盘存储系统之上构建固态硬盘缓存系统^[19,20],相比于使用 RAM 用作缓存系统,拥有更大的缓存空间、更高的缓存性能。当 SSD 作为缓存的存储介质时,由于 SSD 的硬件特性带来了一些新的问题。

第一,SSD 在使用 LRU 等替换算法时,被淘汰替换的数据可以在 SSD 的任意位置,且并不像 FIFO 那样是连续存放的,这就会产生大量的随机写入。对于 SSD 而言,随机写入的性能较低,另外小文件的存在还会带来写放大的问题。由于这样的原因,早期在 Facebook 的 SSD 图片缓存中使用的是 FIFO 缓存替换算法。2015 年,Linpeng Tang 等人^[21],提出了 RIPQ 框架尝试解决在 SSD 缓存中使用高级缓存替换算法时,对 SSD 随机写入的问题。RIPQ 的主要思想是用 RAM 暂存数据,并把多个小文件合并成一个块写入,从而减小对 SSD 的随机写入。RIPQ 借鉴了 GDSF 的思想,同时考量了文件的大小和访问频率,将具有相似热度和大小的文件聚集在一起,这些数据更有可能在相近的时间内被淘汰,从而进一步减少对 SSD 的写入和擦除。

第二,SSD 用作缓存时,密集的写入导致寿命缩短,这也是本文主要解决的一个问题。目前,大多数关于延长 SSD 寿命的研究主要集中在以下方面:SSD 在设计时,如何在性能与寿命之间进行平衡^[22];优化 SSD 的垃圾回收机制^[23,24]和磨损均衡机制^[24-26];构建基于内容局部性的 FTL 层^[27,28];提高纠错编码(ECC)的鲁棒性^[29,30];采用缓存来减少 SSD 写入^[31,32];设计更优的 I/O 调度器^[33];使用 TLC/MLC/SLC 混合的固态硬盘^[34]。这些研究主要是基于 SSD 的硬件特性,解决普遍场景下 SSD 面临的问题,而没有针对用作缓存时进行探究。Sai Huang^[35]等人注意到了这一点。他们提出了 LARC 算法,它由一个存储实际数据的 Content Cache 和一个只保留块元数据的 Ghost Cache 结合起来。缓存预热阶段,与一般的 LRU 算法相同,都需要等待数据将缓存填满。之后进入缓存的正常工作阶段,当一个新的文件到来时,并不会立刻进入到 SSD 缓存(Content Cache),而是保留元数据到 Ghost Cache,等到再次访问时,才真正写入到 SSD 缓存。可见,Ghost Cache 保留了最近只访问一次文件的元数据信息,用来避免这

些文件对 SSD 缓存的无效写入。Content Cache 和 Ghost Cache 都使用 LRU 算法来淘汰旧的数据，Ghost Cache 的大小是动态变化的，当 Content Cache 中的文件被命中时，意味着 SSD 中保留的是当前较为有用的数据，Ghost Cache 的空间将减小，降低新文件进入 SSD 的可能性。反之，则说明 Content Cache 中的数据热度降低，通过增大 Ghost Cache 的空间，来提升新文件进入 SSD 的可能性。由于每次文件进入 SSD 缓存（Content Cache）都需要先通过 Ghost Cache 的考查，只有再次被访问才会写入 SSD，所以减少了对 SSD 的大量写入。最近，Assaf Eisenman^[36]等人设计了一个 Key-Value 缓存系统，他们预估热点访问对象，将其保留在 SSD 缓存中，从而避免对这些数据的重复写入。Yu Zhang^[37]等人针对于云存储系统设计了一个懒惰替换算法，当缓存中的数据足够旧时才会淘汰，从而达到减少 SSD 写入的相同目的。

本文的研究与 LARC 算法所要解决的问题类似。相较于 LARC，LARC 对所有的文件都会阻止其第一次写入，这样做会降低缓存的命中率。而本文的研究工作对每个文件分别做出判定，不仅能降低对 SSD 的写入，同时还能保证缓存的命中率。

1.2.3 图片热度预测

近年来，社交媒体内容的热度预测吸引了广大研究者的兴趣，Lee Breslau^[38]和 Qi Huang^[15]等人，证明了云环境中对象的访问模式呈现类 Zip-f 或帕累托分布。目前大部分的研究主要是预测文本内容的流行度^[39-41]，也有一些研究是对视频的流行度^[42-44]进行预测，对图片的流行度的研究相对较少。下面简要介绍一些对图片流行度预测的重要文献。Aditya Khosla 等人^[45]指出用户的社交信息对于指导图片的流行度预测具有十分重要的意义。他们分别以用户的社交信息、图片的影像特征及前两者相结合的三种预测方案对 Flickr 上图片的浏览量做出预测。社交信息主要包括用户的好友数，拥有的图片数量，及图片的平均浏览量；图片的影像特征主要包括低层次（颜色，纹理）和高层次（物体）的图像特征。其预测结果与实际值达到了 0.81 的相关性。Francesco Gelli 等人^[46]在这项工作的基础上，引入了情感特征，分析了不同情感对于图片流行度的影响。不同于他们的研究，本文的研究工作更加关注于冷文件，而不是热文件，冷文件和热文件之间有大量的温数据，它们与冷数据相似，使得冷文件更加难以区分。值得一提的是，Linpeng Tang 等人^[47]利用预测结果改善了实际的计算机系统，通过对 Facebook

上视频的流行度预测，来决定哪些视频需要进行高清晰度的编码处理，替代了原来盲目的处理方案，减轻了系统计算与存储的压力。

1.3 研究内容

基于对社交网络中图片缓存系统的分析，发现存在着大量的只访问一次的图片。只访问一次的图片进入缓存后不会被再次访问，对缓存而言写入这样的图片就是无用的。故此，本文提出了一次访问过滤的缓存策略，该策略阻止只访问一次的图片进入缓存，能够有效地减少对固态缓存的无效写入，延长固态盘的寿命，提升缓存空间的利用率及缓存系统的性能。为了实现这一目标，首要的问题就是需要对图片未来的访问情况进行预测。以往的文件流行度预测研究多是基于文件的历史访问信息^[48-50]实现对热文件的预测，而本项研究的目标是找出只访问一次的文件，无法依赖文件的历史信息。这使得用现有的方法进行预测时，预测的准确性难以保证。同时由于社交网络中图片缓存服务的多样性，用户群体的广泛性，导致缓存系统的工作负载是动态无规律变化的，使用一种确定的方式进行预测并不现实。经过多次的尝试，最终发现机器学习方法能够很好的帮助我们完成预测任务。针对于缓存系统，本文还设计了几种保障机制用于提升预测质量。

本文内容组织如下：

第一章为绪论部分，主要介绍了高效固态缓存的研究背景，并简要阐述了相关国内外研究现状，及当前研究存在的不足。第二章主要说明本课题研究所涉及到的相关技术，介绍了基于闪存的固态硬盘技术、缓存技术及用于预测的机器学习技术。第三章分析图片缓存系统的主要特点及问题，提出本文的核心研究思路，阐明一次访问过滤缓存系统的设计方案。第四章陈述如何设计一次访问预测的分类系统，介绍怎样运用机器学习方法完成一次访问的分类预测并优化缓存系统。第五章对设计的缓存系统进行全面测试，验证其在 LRU、ARC、S3LRU、FIFO、LIRS 等不同缓存替换算法下的命中率、SSD 写入量、响应时间等。第六章对本文的研究工作做出概括总结，并展望今后的工作方向。

1.4 本章小结

本章简要介绍了课题的研究背景，阐述了目前 SSD 缓存系统所面临的主要问题，简要说明了社交网络中图片缓存系统工作特点，提出了过滤只访问一次的图片的设想。就这一目标，从缓存替换算法、固态缓存系统、图片热度预测三个方面介绍了国内外的相关研究现状。接下来简述了课题的研究内容，最后对本文的组织结构进行了说明。

2 相关技术介绍

在第 1 章中，我们提出过滤一次访问缓存的构想。本章就实现这一目标的所依托的技术作简要陈述。分别对缓存技术、固态硬盘技术和机器学习技术做简要介绍。缓存是本文所研究的对象，2.1 小节将介绍基本的缓存原理及目前常见的缓存替换算法。固态硬盘是缓存的存储介质，2.2 小节将阐述其基本结构和特点，分析其使用时会遇到的问题。机器学习是预测是否是一次访问图片的方法，2.3 小节将简要说明基本的概念，并描述本文所用到的预测方法。

2.1 缓存技术

缓存是存储体系统结构中，用来提升性能的常用手段，它用高速存储设备存储将来可能访问的数据，等到以后被访问时便能快速的响应。在计算机的体系统结构中缓存随处可见。在中央处理器（CPU）上，存在 L1，L2，L3 多级缓存，用来缓解 CPU 与内存之间的速度不匹配的问题；操作系统使用页面缓存将硬盘上的文件缓存在内存中；浏览器使用缓存来加速网页的访问。缓存作为一种行之有效的性能加速手段被广泛地应用于各类计算机架构中。

2.1.1 缓存的基本原理

在存储体系结构，CPU 的一个时钟周期约为 0.4ns ，访问寄存器的时间约为 $0.2\sim 0.5\text{ns}$ ，而访问内存的时间为 $50\sim 100\text{ns}$ ^[51]，访问为普通机械硬盘的时间则需要更久，约为数毫秒。CPU 访问内存或者硬盘都需要等待大量的时间，这便是常说的存储墙^[52]问题。为解决这一问题，出现了缓存机制，它使用高速存储设备存储部分来自低速存储设备上的数据，等到数据被访问时，就能及时响应访问请求。缓存不仅可以用于提升数据的读取性能，也可以用于加快数据的更新和写入，数据在写入缓存后即可响应，提高了响应的速度，数据的真正写入可以推迟完成或者在后台进行。对于存储设备而言，还可以利用缓存将多次小写合并成适合设备的一个大写（块级或页级），减少写入操作的次数。

缓存的本质是以用昂贵的高速存储介质换取更高的访问性能，出于成本原因缓存

容量必定是有限的，且通常比下层的低速存储设备的容量小得多。以普通个人电脑为例，通常内存（DRAM）的容量为 4GB~16GB 不等，而 CPU 的片上缓存（SRAM）通常只有 6MB 左右，仅为内存的千分之一。正因为缓存无法存储海量数据，所以通常需要尽量保存热点数据，以更多的提升访问性能。缓存在大型网站架构中随处可见，在靠近用户侧使用 CDN 缓存网站的静态内容；在机房中心部署反向代理缓存机房中的服务器数据；在应用服务器端，使用服务器本地缓存和分布式缓存加速用户请求处理过程，减轻后端数据库的负载压力。

缓存在大多数情况下都能以较低的代价，取得不错的性能提升。大部分数据的访问遵从二八定律，即 80% 的请求访问的是 20% 的热点文件。因此缓存只需保留热点文件，就能吸收大部分的访问。通常，缓存选择热点文件的方法是根据数据的访问频率，或者访问的局部性原理，这些局部性原理包括时间局部性，空间局部性^[53]，甚至语义局部性^[8]。时间局部性是指，最近访问的数据很有可能被再次访问。空间局部性是指与访问的数据存储在相邻位置的数据很有可能被访问。语义局部性是指一个数据 A 被访问后接下来很可能访问相关数据 B。

对于拥有缓存的系统，通常先在缓存中查找数据，如果找不到的话，意味着发生了缓存不命中，系统需要继续从下层存储设备中查找数据，并返回结果。缓存并不是万能的，它也有其适用的场景。如果缓存的数据被再次访问，那缓存这份数据是有益的。反之，则是无益的。对于一些访问局部性较低，难以预测热点数据的场景，使用缓存可能是无效的，甚至会降低系统的性能。例如一些数据只读取一次，而再也不使用。使用缓存不仅不能提升性能，反而因为每次都会查询缓存而降低了性能。

2.1.2 缓存的替换算法

缓存通常使用比下层存储设备更加昂贵的存储介质，因此，它的容量是往往是更小的。当缓存没有足够的空间来容纳新的数据时，使用缓存替换算法，选择合适的淘汰出缓存，留出缓存空间，给需要的数据使用。评价一个缓存替换算法的好坏多半使用命中率这一指标，它表示所有的数据请求访问中，有多少被缓存命中，直接从缓存返回结果。根据缓存的命中率，能够推算出存储系统的平均访问时间^[54]。

缓存替换算法尽可能的找到将来可能用到的数据，进而提升缓存的命中率，降低

存储系统的访问时间。一个理论上最优的算法——Belady，即是在预知数据将来的访问情况时的缓存策换策略。Belady 算法在发生替换时，每次选择缓存中将来最久才会用到的数据进行淘汰，这样缓存中留下的数据都是比淘汰数据更短时间内会用到的，因此 Belady 算法是最优的缓存替算法。Belady 算法虽然是最优的缓存替算法，但它要求已知将来的访问情况，这实际上并不现实，因此多数情况下，它并不是一个确实可用的缓存替算法。更多的时候，Belady 算法给出了缓存系统所能达到的理论最优命中率。实际的缓存替算法，多是根据数据的某种特征属性对其今后的访问情况进行预估，判定哪些数据更有可能在将来被访问，从而选择哪些数据留在缓存中，哪些数据被淘汰。下面介绍几种常用的缓存替换算法。

RR(Random replacement): 发生替换时随机选择要淘汰的数据，因此不需要保存任何关于数据的信息，十分简洁，被用在 ARM 处理器中^[55]。

FIFO(First in first out): 类似于排队的策略，最早进入缓存的数据，最早从缓存中淘汰出去。这种缓存替换算法实现简单，数据顺序布局，对存储设备不会产生随机写入的问题，早期的 SSD 缓存多采用这种算法，它比较适用数据访问均匀分布的场景。

LRU(Least recently used): LRU 记录最近用到了哪些数据，以及何时用到它们。每次替换时，选择最久没有用到的数据。LRU 主要从数据的访问时间维度来考虑，它基于这样的假设，最近被用到的数据将来更有可能被用到。LRU 在大部分的场景都能表现出不错的性能。

MRU(Most recently used): MRU 算法与 LRU 正好相反，Chou 和 DeWitt^[56]发现当文件被重复性的循环访问时，MRU 是最好的替换算法。MRU 适用于数据随机访问，或者重复性扫描的场景。

LFU(Least-frequently used): 记录数据的访问频率，每次替换时，淘汰最小访问频率的数据。它与 LRU 十分类似，不同之处在于 LFU 考虑的是频率维度，而 LRU 考虑的是时间维度。

以上几种缓存替换算法采用一种单一的缓存替换原则，还有一些缓存替算法与多种因素，如 SLRU^[1], LIRS^[2], ARC^[3]等。

SLRU(Segmented LRU): 它将缓存分成多个段，为了简化，在这里仅对 S2LRU 作简

要介绍。S2LRU 将缓存分成了试用段 (probationary segment) 和保护段 (protected segment)，试用段只有一个，而保护段可以有多个，例如 S3LRU 有两个保护段。一个新的文件首先进入试用段，当其被再次访问时，便认定它当前是可能是一个较热的数据，它将进入保护段。因此保护段中保留的是访问频率大于等于 2 的数据。当保护段空间不足时，保护段采用类似 LRU 的机制将最久未被使用的“热数据”降级成冷数据，重新进入到试用段。值得注意的是，保护段的空间是有固定限制的，而试用段的空间没有确切的限制，对于试用段空间的唯一限制是保护段的数据加上试用段的数据总大小不能超过缓存空间。在 SLRU 算法中，保护段保留了访问频率较高的数据，而试用段则同 LRU 一样保留了最近被使用过的数据，因此这一算法结合了时间局部性和访问频率两种特性。

ARC(Adaptive replacement cache): ARC 算法结合了 LRU 和 LFU，不断在两种算法之间寻找平衡点。ARC 算法与 S2LRU 类似，将缓存分成两个段， T_1 用来存放访问了一次的文件，类似于试用段。 T_2 用来存访问了多次的文件，类似于保护段。与 S2LRU 不同的是，两个段的大小是根据当前的负载状况动态调整的。ARC 算法分别为 T_1 和 T_2 设计了一个 Ghost cache，即 B_1 和 B_2 ，分别用来保存从 T_1 和 T_2 中淘汰数据的元数据信息。当数据被 B_1 命中时，说明负载表现出时间局部性，增大 T_1 的大小。类似地，若被 B_2 命中，则增大 T_2 的大小，减少 T_1 的大小。ARC 算法，能够比较好的避免只访问一次的文件对缓存的污染，被用于 ZFS 文件系统。

缓存替算法被广泛研究了多年，有很多方案。缓存替换算法在设计和选择时，还要根据其场景，选择合适的缓存替换算法，并在算法复杂度和命中率之间进行权衡。如操作系统在使用了时钟策略^[53]，以较小的开销达到接近 LRU 的性能。

2.2 基于闪存的固态硬盘技术

固态硬盘，即 Solid-state drive (SSD)，是一种固态存储设备，它使用集成电路器件来存储数据。1991 年闪迪公司 (SanDisk) 发售了第一款固态硬盘 SSD，容量只有 20 MB，售价昂贵，约为 1,000 美元。近三十年来，固态硬盘技术不断发展，现今一个小小的手机上都有高达 256GB 的存储空间。如此小的体积，却能有如此高的存储空间，这种固态

存储介质正是闪存。除了基于闪存的固态硬盘，还存在一些基于其他的存储介质的固态硬盘，主要包括：（1）基于 DRAM 的固态硬盘，由于 DRAM 掉电后数据会丢失，所以基于 DRAM 的 SSD 通常在内部配备有电池，或是外接电源。（2）闪存与 DRAM 混合，被称为 NVDIMM。当电源关闭时，数据从 DRAM 拷贝到闪存；当电源恢复时，数据又从闪存恢复到 DRAM。由于 DRAM 价格昂贵，空间较小，且数据在掉电后会丢失，目前市面上主要流行的是基于闪存的固态硬盘。

闪存主要分为两种类型，NOR 和 NAND，皆是由 Fujio Masuoka 发明。它们都使用浮栅场效应管（Floating-gate MOSFET）来作为基本的存储结构，都是非易失存储器件。NOR Flash 可以进行随机存取，读取数据方式与 RAM 类似。NOR 支持程序在原地执行（XIP, eXecute In Place）^[57]，即程序不用读取到 RAM 后执行，而是直接在闪存内运行，适具有较高的读取性能，适合用来存储程序的二进制代码。NOR 每次擦除/写入都是以块为单元进行，块的大小可为 16~128KB^[58]。NOR 的写入和擦写速度较慢，主要用于储存不需要经常更新的程序，如 BIOS 程序。NAND 不能像 NOR 那样按照字节随机寻址，而是以页为单元进行读写，但其拥有更低的成本、更高的擦除/写入性能以及更大的存储密度，因此被更加广泛的用于数据存储。NAND 逐渐成主流的固态硬盘存储介质。

NAND 闪存以页（Page）为单元进行读写，一个页通常的大小为 2KB 或 4KB。多个页组成一个块（Block），块是 NAND 擦除的基本单位。一个块通常包含 64 到 256 个页。多个块构成一个分组（plane），分组内可以使用寄存器来提高访问性能。多个分组构成了一个晶圆（die），一个操作可以在特定的分组间并行执行。多个晶圆构成了最后的 NAND 闪存芯片。按照存储单元容量的不同，NAND 闪存还可以分为 SLC(Single-Level Cell)、MLC(Multi-Level Cell)和 TLC(Triple-Level Cell)，分别表示每个存储单元可以存储 1bit, 2bit 和 3bit 信息。其中 SLC，性能最好，也更为可靠，TLC 通常具有更大的存储空间，成本也更低，但寿命相对较短。NAND 闪存支持三种基本的操作，即读、写和擦除。读写以页为基本单位，而擦除以块为基本单元。不支持数据重写，如果要更新数据，必须先要对整个块进行擦除，然后再写入新的数据。NAND 闪存写入操作比读取慢，擦除操作更慢。对 NAND 闪存的写入和数据更新会带来许多问题，主要包括

以下几点:

(1) 速度慢。典型的 SLC 读取数据的时间为 20us, 而写入(Program)却需要 200us, 擦除更是需要 2ms^[59]。写入速度比读取速度慢了 10 倍, 擦除的速度又比写入的速度慢了 10 倍。

(2) 有限的写入/擦除次数, 相对于 HDD 寿命更短。随着写入/擦除次数的增多, 当达到一定阈值后, NAND 闪存中的浮栅晶体管将失去保持电荷的能力, 闪存的可靠性降低, 数据的正确性难以保证。一般来说, SLC 可以承受约 10^6 次写入/擦除, MLC 可以承受约 10^5 次写入/擦除^[59]。而对于 HDD 的而言, 写入次数没有上限。一方面为保证 NAND 闪存中数据的可靠性, 需要设计容错性更高的纠错编码(ECC)和更加复杂的读写电路。另一方面, 为避免对同一个存储单元集中地写入/擦除, 需要设计良好的磨损均衡(wear leveling)策略, 尽量保证闪存中的存储单元有着均匀的写入/擦除次数。

(3) 随着写入次数的增多, NAND 闪存中的无效页不断增加, 需要定期进行垃圾回收(garbage collection), 释放这些无效页占用的空间。由于擦除操作只能以块为单位进行, 而含有无效页的块通常存在其它的有效页。因此, 需要先将块中的有效页拷贝到空闲块, 然后再对含有无效页的块进行擦除。这样的操作必然会对闪存的性能带来一定影响, 同时有效页中的数据被重复写了多次, 在闪存内的实际写入量大于上层应用的写入量, 引发了闪存的写放大问题。

综合以上三点, 大量的写入操作, 对基于 NAND 闪存的固态硬盘来说是极为不利的。正如在 1.2.2 节如介绍的那样, 对于 SSD 缓存需要尽量避免对 SSD 频繁地写入。

2.3 机器学习技术

古时诸葛孔明顺东风草船借箭, 诸葛先生知天文地理, 料世如神, 成为一段千古佳话, 被世人所敬仰, 于蜀国筑武侯祠, 供后人顶礼膜拜。在草船借箭的例子中, 诸葛先生能够根据一些经验规律, 预知未来的天气。人们也总是从环境中学习, 构建自己的观念, 总结一些规律, 去预测未来可能发来的情况, 朝着更好的方向努力。那么, 计算机能不能完成类似的任务。机器学习, 正是出于这样的研究目的, 它由 Arthur Samuel 在 1959 年提出^[60]。机器学习是指计算机系统运用统计学模型和算法, 在没有确切指令的

情况下，运用模式和推理来完成某些任务。机器学习被视为人工智能的一个分支。通常，机器学习算法需要建立一个样本数据的数学模型，称为“训练模型”，以便在没有明确指令的情况下，做出预测或决策。机器学习算法被广泛应用于各种应用中，如电子邮件过滤、网络入侵的检测和计算机视觉。在这些场景下，很难制定明确规则来完成这些任务。机器学习与计算统计学密切相关，计算统计学致力于运用计算机进行预测。机器学习技术以统计学和数学优化研究方法为基石，拥有多种方法和模型来描述现实系统，可以很好地帮助我们预测图片短期内是否只访问一次。

机器学习的学习任务可以分为几大类，包括监督学习，半监督学习和无监督学习等^[61]。在监督学习中，算法根据输入样本以及事先给这些样本打好的标签分析出输入与输出之间的联系。例如，小孩子记忆九九乘法表，每个数字相乘的情况都是由外界告知的，孩子们根据这个乘法规律就可以完成很复杂的乘法。类似地，在监督学习中，样本的期望结果是由外界告知的，而输入输出之前的规律由机器学习模型来表达。根据标签的类型，还可以将任务分为回归和分类两种。对于回归任务，标签是连续值，数据空间可能很大，例如估计一个商品的价格。而对于分类任务，标签是离散值，局限于有限的几个类别中，例如判定西瓜是好是坏。在半监督学习中，部分样本的标签是没有的。在无监督学习中，样本是没有标签的，算法需要找到数据之间的结构和模式，例如把样本分成不同的类别，或是进行数据降维等工作。

在本文的研究工作中，需要估计一张图片短期内是否只访问一次。是一个典型的分类学习任务。本项研究主要用到了决策树算法，现简单地对这一算法做一个基本的介绍。决策树与二叉树类似，回忆二叉树查找一个元素 x 的过程^[62]：从根结点开始，如果 x 大于根结点上的值，则在右子树上继续进行查找；如果小于，则在左子树上继续查找。决策树把这种大于小于的条件换成了更一般的条件。如果满足一定的条件，则在相应的子树继续查找。决策树的叶子节点能够告诉我们输入数据对应的类别。

决策树的孩子节点可以有多个，不过在实现时可以将其转化成普通的二叉树。决策树分类的过程，可以看作是一系列条件的组合，例如同时满足条件 1、2、3 时为类别 A。决策树的关键在于其树的构建过程。决策树的构建过程是一个不断递归的过程^[63]，首选择最优属性 a^* ，然后根据 a^* 上每个可能的取值创建分支。对于每个分支，创建当前

条件下的样本集 D_v ，如果 D_v 中所有样本属于同一个类别或为空集，则可生成叶子节点。反之则重复上述过程。

2.4 本章小结

本章首先介绍了缓存的设计目标和基本原理，接着说明了常见的缓存替换算法，总结每种替换算法的特点和适用场景。随后介绍了固态硬盘技术，阐述了闪存芯片的组织方式和主要特点，并重点分析了闪存的写入和擦除将带来的各种问题，包括速度慢，写入/擦除次数有限，需要进行垃圾回收等。这些指导我们在设计 SSD 缓存时需尽量避免写入。最后介绍了在预测什么样的图片只访问一次时所使用的机器学习技术。

3 一次访问过滤的缓存系统

本章将对社交网络中的图片缓存系统做出简要介绍。首先分析该系统的主要特点与问题，面对系统目前存在的困境，提出了一次访问过滤的缓存策略，随后介绍基于该策略的缓存系统，最后给出一访问图片的判定标准。

3.1 图片缓存系统

3.1.1 图片缓存架构

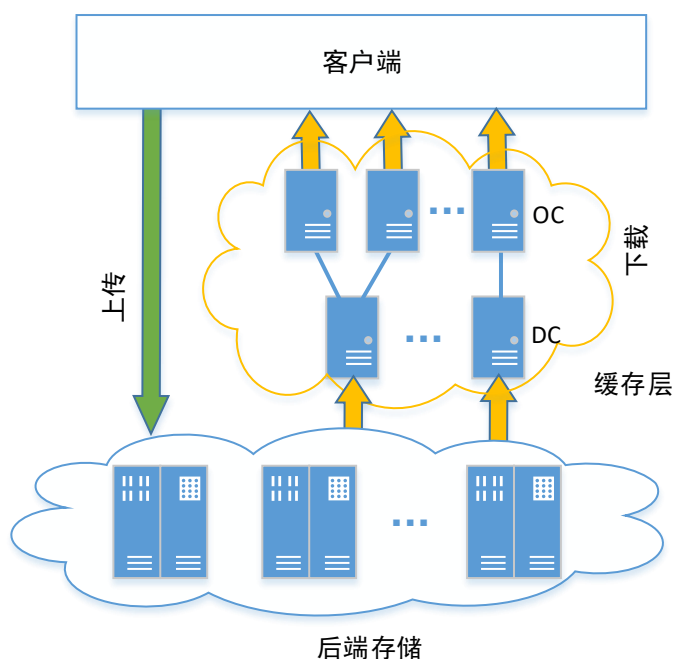


图 3-1 QQPhoto 图片缓存架构

QQ 是极受欢迎的即时通讯软件，其中 QQ 相册是 QQ 用户的照片展示平台。截止到 2017 年底，QQ 相册存储超过 2 万亿张图片，约有 8.5 亿活跃用户，每天有 500 亿次访问量。QQPhoto 作为 QQ 相册的后端照片存储系统，其系统架构如图 3-1 所示，QQPhoto 最突出的特点是上传和下载通道相互之前分隔开来。之所以这样设计，是因为下载流量是上传流量的数千倍。当上传和下载在同一个通道时，巨大的读写流量差异将使系统设计变得十分困难，难以满足用户大量的访问请求。因此将上传和下载通道彼此分离，避免了读/写之间的相互干扰。

正如前文所述，下载流量比上传流量大得多，下载通道的性能对用户体验有着至关重要的影响。为了满足下载通道的高性能需求，QQPhoto 在下载通道上构建了缓存。下载通道上的缓存分为两层，一层是靠近用户侧的缓存服务器，称为 OC(Outside Cache)，另一层是在数据中心构建的缓存服务器，称为 DC(Datacenter Cache)。OC 靠近用户，负责迅速响应用户的访问要求，降低访问延迟。而 DC 位于数据中心，其作用是减轻用户的数据访问对后端存储系统的压力，提升服务性能。两层缓存服务器都配备了 SSD，它们构成了分布式缓存系统来处理海量的图片访问请求。

3.1.2 图片缓存系统的主要特点

QQPhoto 的缓存系统效率对 QQ 相册的性能至关重要。为了优化相册缓存，有必要进一步深入了解当前的缓存系统，以发现有待改进的方面。我们收集了约 9 天的 QQ 相册访问日志，共计 1,481,617,402 张图片和 5,856,501,598 次访问。通过分析这一日志，观察到了 QQ 相册工作负载的以下几个主要特点。

特点 1：大量无效写入

通过对访问日志的统计分析，发现约有 910,568,705 张图片仅仅只访问一次，占总数的 61.5%。这些一次访问的图片占整个访问量的 15.5%，而其它的图片在第一次访问时也会发生强制访问缺失，约占 10%，所以即使缓存空间无限大，缓存命中率上限也是 74.5%，这可以称为“天花板”现象。

在传统的缓存方法中，如果一个访问请求没有命中，那么访问的文件将立即从后端存储写入缓存，即便将来可能并不会用到这一份文件。通过对访问日志的分析，发现有相当多的一次访问图片，这些图片写入 SSD 缓存是不必要的，因为其不会被再次访问，也就不会发生缓存命中。不必要的缓存写入可能会导致两种负面影响：首先，它们造成了对缓存空间的污染，被一次访问图片占用的缓存空间原本可以用来缓存其他有用的数据以提高缓存性能。更进一步，它们加速了 SSD 的磨损，缩短了 SSD 的寿命。

特点 2：缓存替换算法和缓存空间的影响有限

除了上述关于应用日志的分析，本文还研究了缓存本身。通常，缓存替换算法和缓存大小都是影响缓存性能的关键因素，那么这两个因素在这里表现又如何呢？我们利用收集的日志进行缓存仿真实验，采用目前流行的缓存替换算法，包括 LRU, S3LRU,

ARC 和 LIRS。此外，Belady 算法，用来给出缓存系统性能的理论上限以及潜在的改进空间。实验结果如图 3-1 所示，其中 x 和 y 轴分别表示缓存容量和命中率。

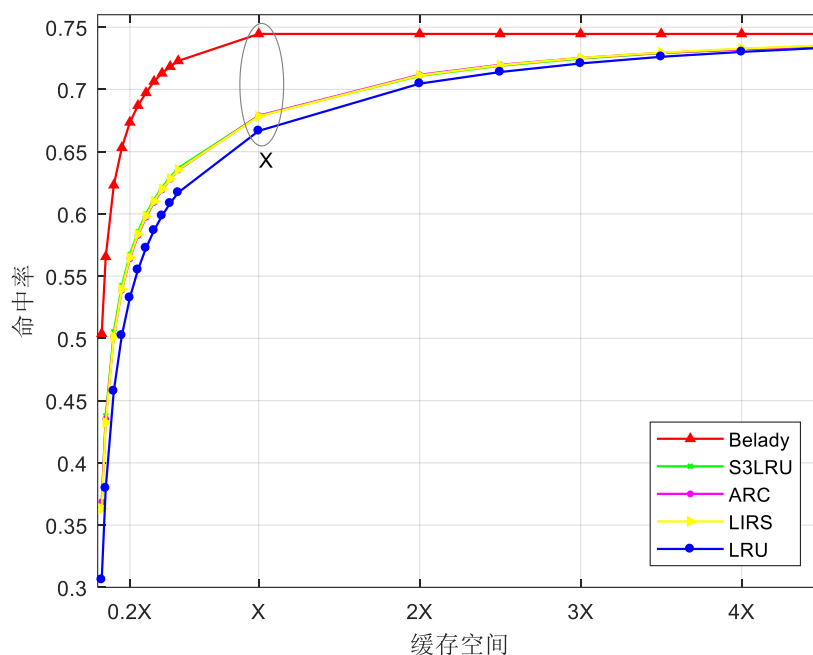


图 3-2 不同缓存空间下的命中率

我们发现算法的命中率存在一个拐点 X ，当缓存空间超过 X 后，Belady 算法的命中率保持稳定，而其他缓存替换算法的命中率增长也逐渐放缓。其次，除了 Belady 算法，其他算法在命中率上的差异几乎可以忽略不计，特别是 S3LRU，ARC 和 LIRS 几乎都有相同的命中率，它们都优于 LRU 约 1%。当缓存空间在点 X 处时，Belady 算法和其他算法的命中率差距约为 9%，当缓存空间增加到 $4X$ 后，命中率差距逐渐减少到 4%。综上，这两点现象意味着试图扩大缓存空间或是选择不同缓存替换算法对于缓存系统性能的影响较为有限。

3.2 策略核心思想

3.2.1 核心思想

相较于日常存储，当 SSD 被用作缓存时，SSD 将面临更大量的写入。如果可以减少对 SSD 的写入、延长其使用寿命，那么存储服务提供商便可降低硬件成本，从中获

益。在缓存性能改进方面，一味地增大缓存空间来提升缓存性能是不切实际的，因此优化性能的关键变成了如何提高缓存的利用率。

在传统的缓存过程中，一旦文件访问没有命中，丢失的文件将被写入到缓存中，这个方案做出了这样的假设：当前文件很快将会被访问。但是，在大规模互联网环境中，用户的访问模式可能变得十分复杂和难以预测，原来的假设可能并不会成立。

在社交网络的图片缓存中，有大量的一次性访问图片。访问这些图片后，它们将进入并驻留在缓存中，却并不会被访问，直到最后它们被替换出缓存。对于缓存系统而言，这些一次访问图片将带来大量写入，并且它们占据了原本可以用来存放更加有用数据的空间，导致了缓存污染，降低了缓存利用率。因此，本文提出在一开始便阻止它们进入缓存系统。特别是，当缓存空间有限，阻止这种写入的好处将更加明显。我们称之为一次访问过滤策略。

3.2.2 主要挑战

为了防止一次访问文件进入缓存，有必要预测当前访问的文件是否是一次性访问文件。这里的预测目标是一次性访问文件，它们以前从未访问过，因此无法使用历史信息。无历史信息的预测往往比基于历史的预测更难。基于历史信息的预测已经被广泛的研究过^[48-50]，而无历史信息的预测只能抓住这些图片的共同特点，在图片第一被访问时就要做出较为正确的决断。此外，云存储服务环境中的服务类型和用户群体是多种多样的，这也意味着工作负载是动态变化的。因此，以某种确定的方式来预测是不太现实的。机器学习在计算机体系结构中的应用是一个新兴的研究领域。它在大规模数据分析方面表现优异，并且能够跟随动态的工作负载更新模型，自动完成特征选择。最近，也出现了将机器学习应用于实现 NVMe 系统中动态均衡策略^[64]的研究。可见使用机器学习来完成这一预测是一个不错的选择。

3.3 缓存架构设计

图 3-3 展示了实现一次访问过滤策略的缓存系统架构，其主要由四部分组成：(1)SSD Cache，(2)后端存储系统，(3)分类系统，(4)离线训练模块。该缓存架构与传统的缓存架构相比，主要增加了分类系统及离线训练模块。分类系统的主要作用是，判定

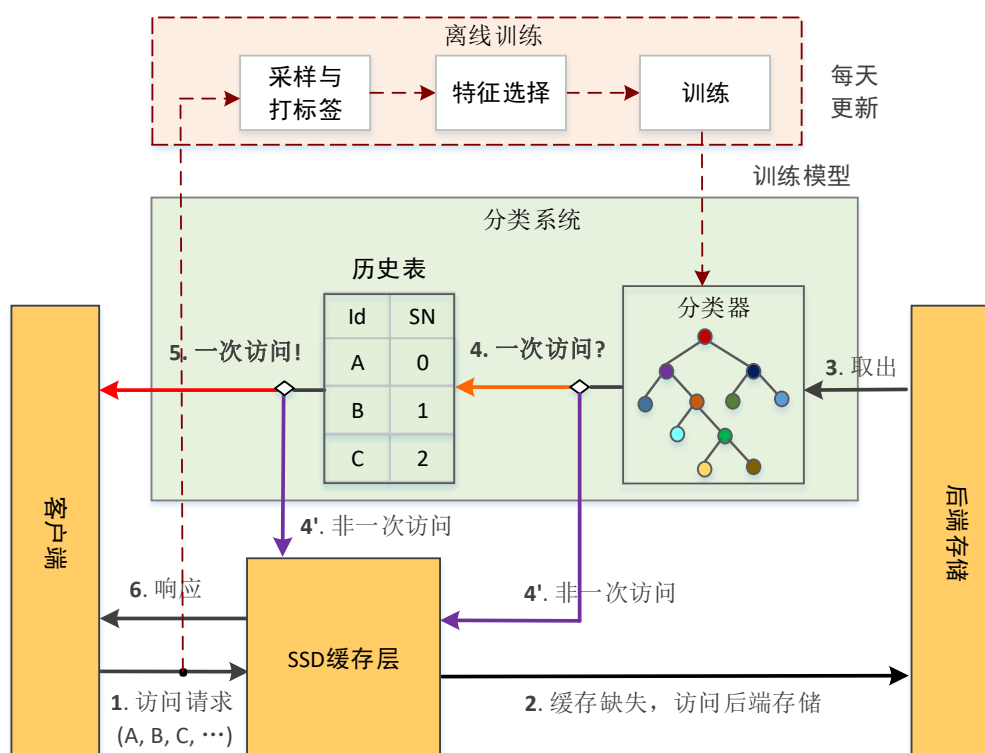


图 3-3 一次访问过滤的缓存架构

即将加入缓存的图片在未来的一段时间内是否会被再次用到。如果图片不会再次用到, 那么图片数据不会进入 SSD Cache, 而是直接返回给客户端, 从而增大 SSD Cache 空间的利用率, 并减少了对 SSD 的写操作。分类系统用于预测的机器学习分类器由离线训练模块周期性地构建, 保证模型能够反映当前的用户访问特性。离线训练模块主要有以下几个功能: 数据采样和打标签、特征选择、模型训练。详细内容将在第 4 章进行介绍。

分类系统由两部分组成: (1) 分类器, (2) 历史表。分类器根据图片相关属性特征作出判断。而历史表则保留着近期被判定为一次访问图片的元数据信息 (详见 4.4.3 节)。当一张图片被分类器判定为一次访问图片时, 分类系统仍有可能根据历史表中的元数据信息将其修正为非一次访问图片。历史表根据最近的访问情况检测图片是否存在误分类, 避免误分类对缓存系统产生进一步的不利影响。这里的历史表仅仅起到了检测反馈的作用, 与无历史信息预测并不矛盾, 预测任务仍然需要在图片第一次访问时完成。整个分类系统偏向于将图片分类为以后将会用到的非一次访问图片, 其原因在于如果这样的图片被误分类将产生多次的缓存不命中, 相对于允许图片写入缓存所带

来的缓存空间消耗，前者的代价要大得多。

如图 3-3 所示，添加分类系统后，一次访问过滤的缓存系统的工作流程如下：

- 1. 缓存接收到来自客户端的访问请求 (A, B, C) 后，如果命中，请求的图片将返回给客户端，处理完成。否则，请执行下一步。
- 2. 如果访问的图片没有被缓存命中，那么访问请求将会被转发到后端存储。
- 3. 后端存储系统返回图片给分类系统。分类系统中的分类器首先根据图片的属性特征预测其是否是一次访问图片。如果是一次访问图片，则转步骤 4。反之，则转步骤 4'。
- 4. 历史表根据过去一段时间内的访问信息，判定其是否是一次访问图片。如果是，则图片直接返回给客户端（步骤 5）。反之，则转步骤 4'。
- 4'. 分类系统判定图片为非一次访问图片，这时将采取传统的方案，即图片写入缓存，转步骤 6。
- 5. 分类系统判定图片为一次访问图片，图片直接返回给客户端。
- 6. 图片从 SSD 缓存返回到客户端。

与传统的缓存系统相比，一次访问过滤的缓存架构添加了分类系统，用来阻止一次访问的图片进入缓存。分类器主要使用离线训练策略更新模型，预测的开销很小，同时能够增加缓存的利用率。

3.4 一次访问判断标准

为了将一次访问图片从缓存过滤，需要很好地判定什么样的图片是一次访问图片，给出一个比较确切的评判标准。一次访问，比较自然的定义是文件指在访问记录中只会出现一次。根据该定义分析图片缓存系统的访问记录，发现：

- (1) 在所有的图片中约有 61.5% 的图片只会被访问一次
- (2) 在所有的访问记录中，对一次访问图片的请求约占有所有访问请求的 15.5%

根据这一定义标准，我们做了假定已知图片是否是一次访问的理论实验。图 3-4 中蓝色线表示 LRU 算法在缓存替换时过滤了该定义所指的一次访问图片时的命中率。可以看出过滤了一次访问图片后对 LRU 算法的命中率有较大的提升。仔细观察图 3-4，

在 2GB 缓存空间下的命中提升不及 4GB 下的命中率提升，这不太符合原本的预期。过滤一次访问图片的目的之一是提高缓存空间的利用率，在缓存空间较为紧张的情况下，缓存空间被有效的利用意味着，应该有更大的命中率提升才对，这样的定义是否存在问题？注意，这里本文所述的 2~40GB 缓存空间，对应实际 200GB~4TB 的缓存空间。

那更为合理的定义应该是怎么样的呢？简洁来说，如果图片驻留于缓存的生命周期内只被访问了一次，那么可以说该图片是一次访问图片。当图片进入缓存后，有一些

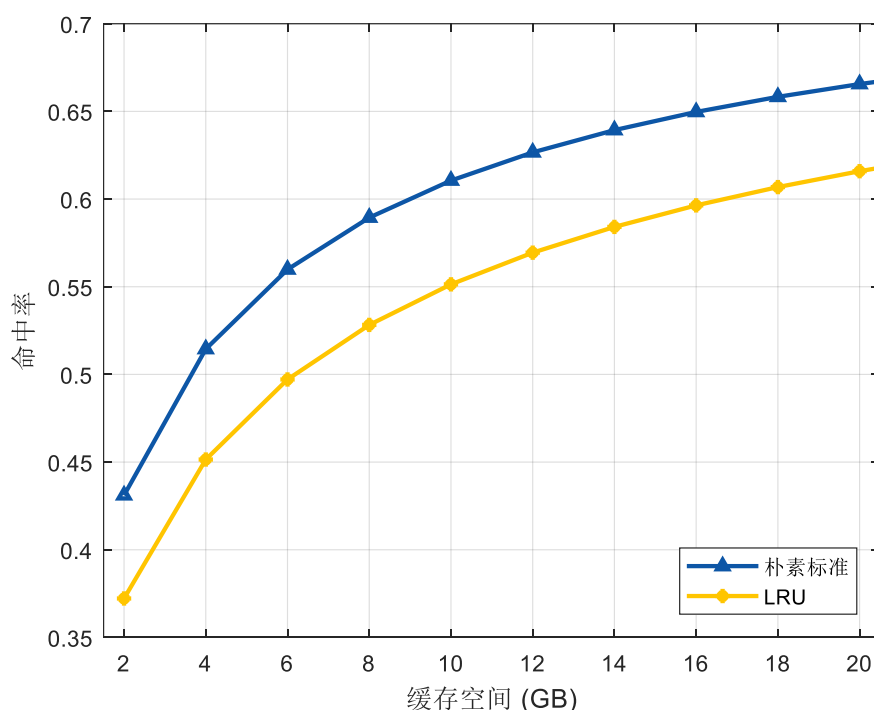


图 3-4 过滤整个访问日志中只访问一次的图片

图片可能短时间内会被再次访问，而有些则不会。而缓存空间终究是有限的，那么存在这样一种可能，即图片进入缓存后没有等到被再次访问就被替换出去了。这样的图片不仅没有带来缓存命中率的提升，同时也增加了对 SSD 缓存的不必要写入。它们就可以被当作一次访问的图片并阻止其进入缓存，那么接下来的关键是如何量化这一标准。

如果一张图片进入缓存后经过 M 次访问后被替换，并且这 M 次的访问中都没有再次访问该图片，也就是说图片再次访问的距离大于 M ，便判定该图片是一次访问图片。接下的问题便是如何求得 M 。

若缓存的命中率为 h , M 次的访问便有 $M(1-h)$ 次不命中, 对于一个已经处于正常工作状态的缓存 (缓存空间几乎用尽), 也就是说会发生 $M(1-h)$ 次替换。以 LRU 为例, 若缓存大小为 C , 图片的平均大小为 S , 一个不会被重用的图片最多经过 C/S 次替换操作后从缓存的 MRU 位置替换出缓存。故有:

$$M(1-h) = C/S \quad (3-1)$$

易得 $M = C/[S(1-h)]$ 。

这样的定义是否就足够了呢? 考虑到最终会过滤这些一次访问图片, 意味着在这 $M(1-h)$ 次不命中的访问中, 其中一部分图片会被过滤掉, 不让其进入缓存。若对这些一次访问图片的请求占总访问请求的比为 p , 那么 M 次访问中最会有 $M(1-h)(1-p)$ 次的访问会引发缓存的替换操作。故有:

$$M(1-h)(1-p) = C/S \quad (3-2)$$

易得 $M = C/[S(1-h)(1-p)]$ 。

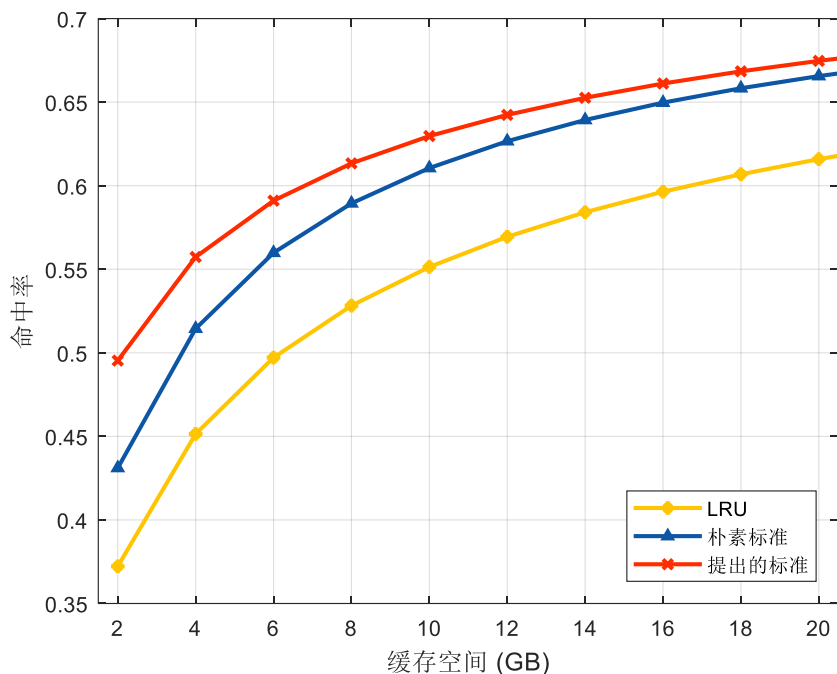


图 3-5 不同定义标准的对比

图 3-5 对比了两种定义下过滤一次访问图片后 LRU 算法的命中率情况, 可以看到新定义在缓存空间的减小后, 对命中率的提升更加明显。下面介绍说明 M 的具体计算

过程。在公式(3-2)中,当 h 为一个固定常量时,随着 p 的增大, M 也越大。而对于图片缓存而言, M 越大意味着一次访问图片的判定条件越苛刻,重新访问的间隔大于 M 的文件越少,也就意味着 p 越小。这样的趋势可以概括为: $p \uparrow \rightarrow M \uparrow \rightarrow p \downarrow$ 。同理,存在一个相反的规律,即: $p \downarrow \rightarrow M \downarrow \rightarrow p \uparrow$ 。

假定 p 初始为 0,对应的 M 可由式(3-2)得出。以这一个 M 作为标准,重放图片缓存的访问日志,计算出一次访问图片的比例 p 。再以此时的 p 计算出一个新的 M ,重复上述步骤,迭代一定次数后 p , p 将收敛到一个常量, M 便可求出。一般迭代次数为 3 次。预估缓存命中率 h ,也可按此方法。

3.5 本章小结

本章首先介绍了 QQPhoto 图片缓存架构,分析了该系统的主要特点,即存在对缓存大量无效的写入,同时缓存空间和缓存替换算法对系统性能的提升有限。这些特点,激发了本文的研究灵感:过滤只访问一次的图片,不让其进入缓存。由此,设计了一次访问过滤的缓存系统,并对这一缓存架构做了简要介绍,描述了其工作流程。最后,提出了一次访问文件的判断标准,以进一步指导构建机器学习分类器,选择什么样的图片进入缓存。

4 一次访问分类预测

3.3 节介绍了一次访问过滤的缓存系统，其核心在于实现一次访问预测的分类系统。本章将详细介绍分类系统设计方案，首先阐述机器学习模型训练时的数据集采样方法，接下来讨论如何选择分类算法，然后介绍如何完成特征选择，最后说明几种分类预测质量的保障机制。

4.1 数据采样

在开始之前，为了完成机器学习分类器的训练，需要一个采样器来完成机器训练时所用数据集的构建。采样器的功能是获得构建分类器时所需要的样本数据并完成对样本数据打标签的工作，它随机地对一些图片观测一段时间，判定其是否为一次文件。采样器以一定的概率 P_{sample} 随机的选择需要进行观测的图片，如果图片的两次访问的距离小于等于 M 则标记为非一次访问文件，反之，若图片两次访问的距离大于 M 或图片被采样器淘汰时仍未被访问，则标记为一次访问文件。若文件在进入采样器后存在多次访问，仅采用这些图片首次访问时的特征作为训练样本，原因有两点：一是，在一次访问过滤的缓存架构中，分类器更多地需要在首次访问时，判断其是否为一次访问文件，被多次访问的图片进入缓存后，不再需要分类器重复地做出判断。二是，在本文即将用到的决策树模型中（参见 4.2 节），每个叶子节点对应着一定条件下的样本子集，若这个子集中哪一类图片更多则更有可能被分类成相应的图片类型。若将图片每次访问时的特征都作为训练样本，则很有可能导致在训练出的决策树叶子节点上，有大量的非一次访问样本都来自于同一个图片，影响分类结果的正确性。最后，采样器观测图片的最大数目为：

$$C_{\text{sample}} = M(1 - h)P_{\text{sample}} \quad (4-1)$$

其中 $M(1 - h)$ 估计了近 M 次访问内新文件的数目，最后再乘以采样率得到采样器的空间大小。采样器使用 FIFO 策略淘汰旧的图片。

4.2 分类算法

4.2.1 分类算法的选择

有许多机器学习算法可以用来预测。选择这些算法需要在预测准确度和算法复杂度之间找到一个很好的权衡。为了实现这一目标，本文对目前主流的分类算法进行了广泛的对比。使用 4.2.1 节中描述的分类器采样了约 7 万条数据。接着将采样得到的数据集分成训练数据集和测试集，使用数据集完成训练，而用测试集来评估分类器算法的性能。表 4-1 给出了各个分类器的性能评估，而图 4-1 绘制了它们的 ROC 曲线。关于机器学习评价指标的含义由表 4-2 和表 4-3 给出。从表 4-1 可以看出，决策树、集成学习算法（如 AdaBoost，随机森林）都达到了良好的分类性能。

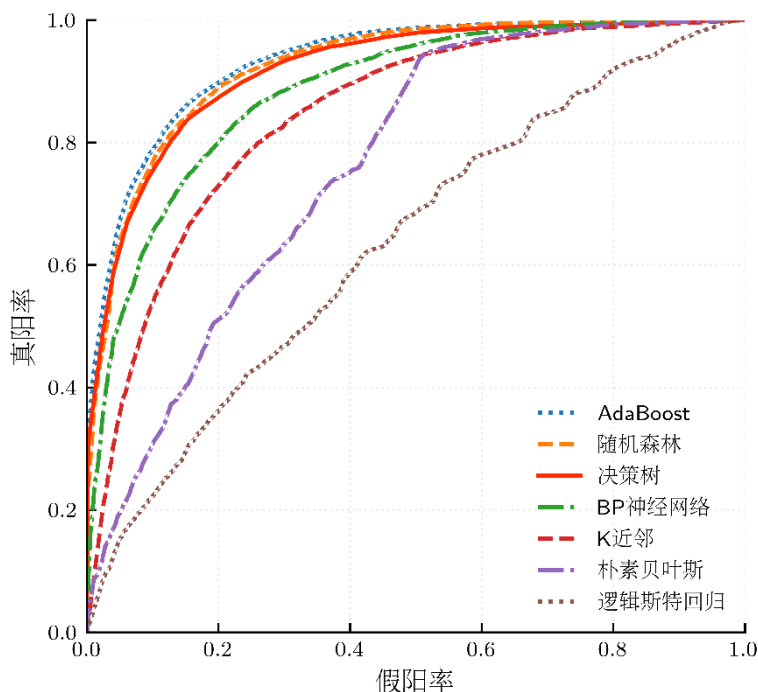


图 4-1 分类器的 ROC 曲线

直观来讲，集成学习算法巧妙地结合了多个决策树，具有更好的分类准确性，是分类算法的一个不错选择。但是，我们发现在使用 AdaBoost / 随机森林等集成学习分类器时，即使基学习器（决策树）的数量增加到 50 个，它们也仅仅能提高 1% 的分类准确性，同时却带来约 50 倍的计算成本。在分类器选择上，本文遵循两个准则：一是具有较高的 AUC，这往往表明算法在假阳率较低的情况下也有不错的真阳率。二是考虑

到缓存系统的性能需求，分类器的复杂性不宜过高。根据这两个准则，本文选择决策树^[63]作为最终分类器，其中分类性能与随机森林几乎相同，但其复杂度要低得多。

表 4-1 分类器的性能对比

算法	准确率	召回率	精度	AUC
AdaBoost	0.842	0.876	0.854	0.854
随机森林	0.847	0.85	0.846	0.846
决策树	0.846	0.841	0.842	0.842
BP 神经网络	0.754	0.879	0.794	0.793
K 近邻	0.76	0.797	0.77	0.769
朴素贝叶斯	0.563	0.991	0.606	0.602
逻辑斯特回归	0.603	0.505	0.582	0.583

表格中的算法可以参阅文献^[61]

表 4-2 预测结果的混淆矩阵

真实情况	预测结果	
	正例	反例
正例	真正例 (TP)	假反例 (FN)
反例	假正例 (FP)	真反例 (TN)

表 4-3 机器学习评价指标的定义

评价指标	定义
准确率 (Precision)	$Precision = TP / (TP + FP)$
召回率 (Recall)	$Recall = TP / (TP + FN)$
精度 (Accuracy)	被正确分类的样本占总样本的比例
ROC 曲线	一条横坐标是真阳率，纵坐标是假阳率
AUC	ROC 曲线下的面积

4.2.2 分类算法的配置

为了避免分类器过度拟合而导致的泛化性能降低，决策树的最大叶节点数目被设定为 30，这大约是特征数量的 3 倍。决策树算法的预测性能与树的高度相关。经过广泛测试，我们发现大多数情况下，树的高度为 5。因此，仅仅需要 5 次比较操作就能完成预测。

4.3 特征提取

4.3.1 特征介绍

(1) 图片所有者的社交信息

活跃好友数：最近与图片所有者互动的用户数量。

图片平均浏览量：即用户所有图片的总浏览量除以其拥有的图片数目。

(2) 图片信息

图片类型：它表征了照片的分辨率和所使用的规格标准。在本文研究的图片缓存系统中有六种等级的分辨率（a, b, c, m, l 和 o）和两种不同的图片规格（即 png 和 jpg，分别用 0 和 5 表示），组合在一起总共十二种照片类型。如图 4-2 所示，不同类型的图片之间的请求数量存在显著差距。其中，m5 的请求数量最多，约占 45%。对于某一种类型的图片而言，其访问概率是相对固定的，但也会随着时间出现一些波动。这说明用户的访问时间也应该作为分类器预测的一个特征。

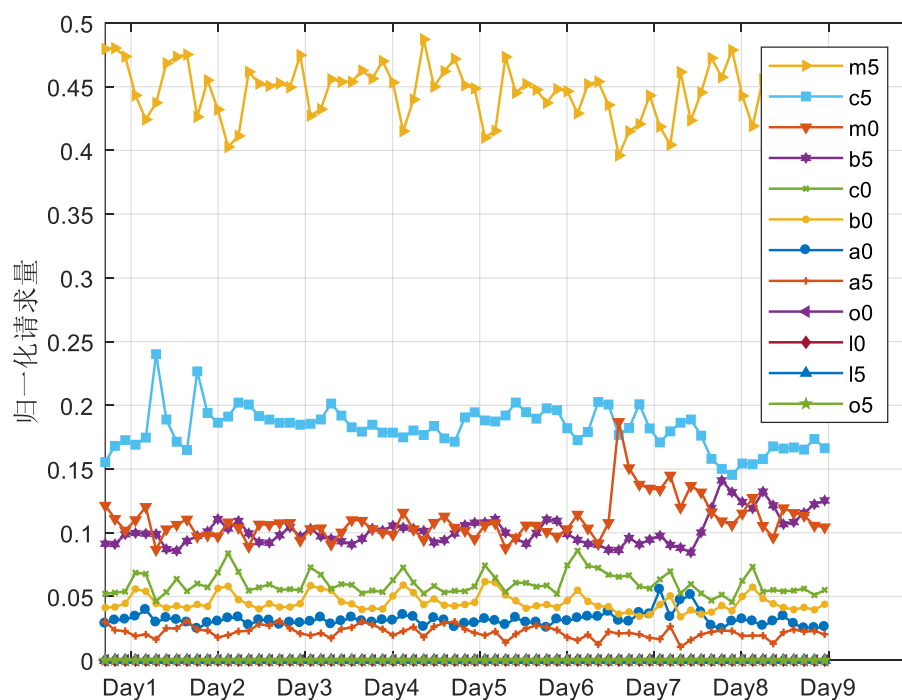


图 4-2 不同类型的图片访问量

图片大小：图片尺寸与图像的分辨率有很强的相关性。一般来说，相同的图像，分

分辨率越高，照片越大。不同分辨率的图片，其访问概率也存在不同。

图片年龄：通过当前时间点和图片上传时间点之间的时间间隔来表示图片年龄。直观上，较新的图片更受欢迎。统计分析实验结果也证实了这一观点。

图片 Recency：当前图片访问时间与其上次访问时间之间的时间间隔。如果之前从未访问过该图片，它将用图片访问时间与其上传时间之间的时间间隔来代替。

(3) 缓存系统的相关信息

终端类型：终端类型主要包括 PC 和手机端。

最近的请求量：系统在最近一分钟内接受的请求数。系统短期内的请求量可以间接反映用户整体的活动频率，更高的请求量表明用户整体较为活跃，相应的每张照片的平均访问概率更高。

访问时间：访问发生在一天中的哪一个时间段。用户经常在相对固定的时间段使用 QQ，通常晚上 8 点是用户访问的高峰，这也意味着不同的时间段有着不同的图片访问概率。

4.3.2 特征预处理

特征预处理包括为样本打标记，类别特征的离散化处理和数值值的处理。3.4 节介绍了使用 LRU 算法时如何判定一个图片是否是一次访问图片。该定义也适用于当前其他缓存算法，如 ARC，S3LRU，FIFO 等。训练样本经过采样器完成样本的标记过程。

图片的特征属性中有两个是类别特征，一是图片类型，它包括 12 种不同的图片类型，二是终端类型。本文所使用的决策树算法，不需要对这样的类别特征作特殊处理，只需要转化为一个量化值即可。12 种类型的照片（a0，a5，b0，b5，c0，c5，m0，m5，o0，o5，l0，l5）一一映射成离散值（1-12）。终端类型，PC 和移动设备分别映射为 0 和 1。

数值值的处理主要针对特征中的访问时间和其它一些表示时间长度的值。社交应用中的工作负载与人类活动高度相关并表现出某种周期性，因此我们将访问时间转化为访问时段，用一天中的小时（0-23）表示。图片的年龄和 Recency 以 10 分钟为基本单位表示。人类作为 QQ 相册图片的使用者，使得图片访问的时间差变为人类活动的时间粒度，因此将这些时间差按 10 分钟为基本单位处理，而不是以秒为基本单位。

4.3.3 特征选择方法

特征选择的目的是从原始的 n 个特征中，选定一个特征子集，仅使用特征子集中的特征进行模型训练，降低训练的难度，避免特征间的相互干扰。最初，所有的 n 个特征形成一个候选特征集，即 $\{a_1, a_2, \dots, a_n\}$ 。将每个特征视为一个候选子集，然后使用信息增益（信息增益越大，对分类越有帮助）来评价这些子集，选择最佳子集，即具有最大信息增益的特征，构建出初始的选定特征集合， $\{a_i\}$ ，并从候选特征集中删除 a_i 。进行下一轮，试着将候选特征集中的每一个特征 a_k 加入到集合 $\{a_i\}$ 中，继续用信息增益评价这 $n-1$ 个特征子集，并选定最佳特征子集，即 $\{a_i, a_j\}$ 。如果选定子集 $\{a_i, a_j\}$ 的信息增益优于之前的 $\{a_i\}$ ，这意味着该轮选定的特征子集用于分类的效果比以前好，那么相应的过程将重复。否则，该过程停止，特征选择完成。基于此方法，最终选定了图片平均浏览量，图片的年龄及 Rencency，图片访问时段及图片的类型等几项特征。

4.4 分类预测质量的保障机制

为了确保预测质量，本文采用代价敏感学习，历史表和每日更新模型三种保障机制。代价敏感学习减少不利的情形发生的概率。在错误分类发生后，由历史表阻止其对缓存系统产生进一步的负面影响。这两种方法提升了预测结果为“是一次访问图片”时的可信度。同时，每天更新模型，以确保分类器表示的模型与当前用户访问模式一致。

4.4.1 代价敏感学习

分类器可能的错误分类情况有两种：一种是将一次访问图片误分类为非一次性访问，即假阴性；另一个则相反，将非一次性访问图片误分类为一次访问，即假阳性。第一种错误将导致缓存空间浪费和对 SSD 的写入，而第二种误分类会导致对该图片后续的访问不会被缓存命中。为降低由误分类带来的影响，本文采用如表 4-4 所示的代价矩阵^[65]。

对缓存而言，由第二种类型的误分类带来的缓存不命中相对于第一种错误带来的代价更高，因此 $v > 1$ 。当缓存空间较低，缓存空间的对命中率的影响作用更加明显，因此第二种误分类的代价相对较低，反之亦然。因此在缓存空间越大时，应该设定较大

的误分类代价 v 。 v 值的选择范围是 $\{1.5, 2, 2.5\}$ ，选择 v 值的方法将在下一节中介绍。

表 4-4 代价矩阵

真正的类别	分类成一次访问图片	分类成非一次访问图片
一次访问图片	0	1
非一次访问图片	v	0

4.4.2 分类器的工作点选择

工作点即决定分类器工作在 ROC 曲线上的哪一个点，它需要在权衡分类准确率（*precision*）和召回率（*recall*）后作出选择。一种常见的评估方法是使用 F_β 函数，其计算方法如公式(4-2)所示。

$$F_\beta = (1 + \beta^2) \cdot \frac{\text{precision} \cdot \text{recall}}{(\beta^2 \cdot \text{precision}) + \text{recall}} \quad (4-2)$$

当 $\beta = 1$ 时， F_β 表示了准确率和召回率的调和平均数。它表示准确率具有相同程度的重要性。当 $\beta > 1$ 时，它意味着召回率比准确率更加重要。当 $\beta < 1$ 时，意味着准确率更加重要。在本文研究的场景中，即使把图片误分类成非一次访问图片也只是采用了传统方案相同的策略，不会造成命中率的降低，仅仅可能导致对 SSD 写入量的增加。相反如果把文件误分类成一次访问文件，不让其进入缓存，将会导致此后的访问出现中缓存缺失，较大的影响命中率。本文认为在缓存系统中保证命中率应当比减少对 SSD 的写入量具有更高的优先级。因此，第一种情形是可以接受的，应该尽量避免第二种情形的发生。这也就要求，被认定的为一次访问图片有较高的可信程度，也就是准确率更加重要。因此，将 β 设置为一个小于 1 的值。

具体的， β 值的选择还要考虑缓存空间。当缓存空间较小时，数据进入缓存后能够驻留时间相对较短。因此，平均而言，低空间下，图片命中次数比高空间下更少。这也意味着，低空间下，误分类非一次访问图片带来的缓存缺失更少，对准确率的要求也更低。因此 β 值在低空间更接于 1，在高缓存空间下更接近于 0。本文设定，在 2GB，20GB，40GB 时， β 分别定为 $1/1.5$, $1/2$, $1/2.5$ 。其余空间下的 β 值借由这三个点确定的二次函数计算。 F_β 函数可以评估不同模型的好坏，从而对代价敏感矩阵中的 v 值作出选择， v 的取值范围为 $\{1.5, 2.0, 2.5\}$ 。注意这里 2GB，20GB，40GB 的缓存空间对应着实

际为 200GB, 2TB, 4TB 的缓存空间。

4.4.3 历史表

3.3 节简要地介绍了历史表, 本小节将对其进行更详细的讨论。历史表对纠正先前的错误分类方面起着重要作用。特别地, 在这里错误分类指的是把非一次访问图片误分类为一次访问, 这将导致图片后续的访问不被缓存未命中。如图 3-3 所示, 历史表使用一个 Hash 表来存储最近被归类为一次访问图片的元数据信息。这些信息为图片的文件名(id), 及最后一次访问时的次序(SN, Sequence Number)。对于历史记录表中的图片, 再次被访问时, 由当前访问次序与图片上次访问时的次序两者求差, 得到两次访问之前的间隔。若小于 M , 这意味着图片上次被错误地分类为一次性访问图片。此时, 为避免这一误分类进一步引发更多的缓存不命中, 分类系统将视其为非一次性访问图片, 并从历史表中删除。历史表使用 FIFO 策略淘汰旧的数据。存储在 DRAM 中的历史表的容量被设置为 $M(1-h)p \times 0.05$, 仅为 SSD 缓存元数据表空间的 2%~5%。

历史表的目的是减少错误分类的影响, 这里是仅指将非一次性访问图片误分类为一次性访问图片, 这通常导致更多缓存未命中, 对于缓存系统而言较为致命。历史表仅在决策树将图片分类为一次访问图片时发挥作用, 如果它告诉我们在不久的过去不曾有错误分类, 则可以将文件视为一个一次访问图片。如果不使用历史表, 系统对一次访问图片的分类精度将降低, 并且缓存系统的命中率也会下降。具体影响将在 5.4 节中进行详细分析。

4.4.4 训练模型更新

我们对第 4.2.1 节中描述的数据集进行了训练, 并使用了代价敏感学习方法构建分类器。分类器的输入即为 4.3.1 节中所介绍的特征, 输出即为图片是否是一次访问文件。将训练得到的分类器配合历史表构建出分类系统, 使用所有的日志访问记录对分类系统进行一次访问预测的分类测试。结果表明分类性能随着时间的推移显著下降, 如图 4-3a 所示, 这是因为该分类器是对用户访问特性作出的概率建模, 而用户的访问特性会随时间发生变化, 分类预测模型存在适用时效。

这个问题有两种解决方案。第一种是使用在线增量学习方案, 实时更新分类器模

型，第二种是仍采用离线学习方式，每隔一段时间更新一次分类器，通常在负载较低的时候更新分类器。本文选择了第二种方案，其原因在于对缓存系统的影响更小。为了确保

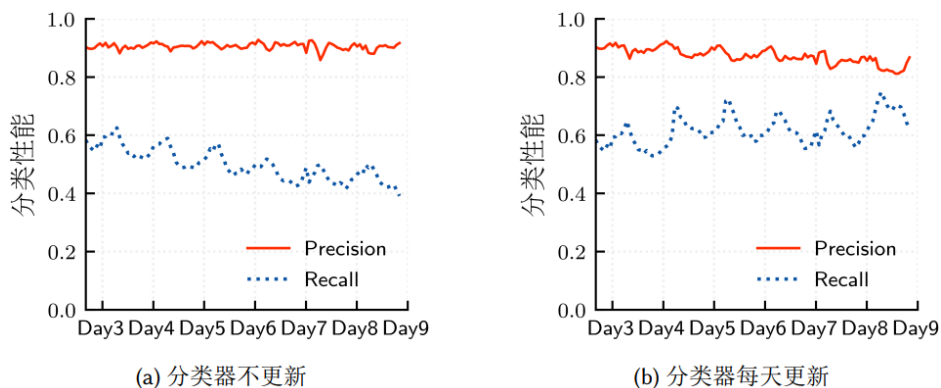


图 4-3 分类器的性能比较

定更新的周期，我们对系统中一次访问请求比例 p 随时间的变化情况进行了分析。分析结果显示， p 以一天为单位周期性波动，分别在凌晨 5:00 和晚上 20:00 达到最高峰和最低值。同时考虑到系统的工作负载，选定在每天请求量较低的凌晨 5:00 更新模型，降低对系统的影响。在凌晨 5:00 时，从采样器获得一天的数据，训练并更新分类器，用于接下来一天的预测。由于 CART 决策树的高效性，整个训练过程只需几分钟便可完成。每天更新模型的分器性能如图 4-3b 所示。对于比与图 4-3a，召回率随时间的变化更加稳定。

4.5 本章小结

本章讲述了一次访问预测分类系统的设计方案，介绍了将机器学习应用于本文设计的一次访问过滤缓存系统时，需要着重考虑的几个方面。首先说明了如何设计一个采样器来获得用于训练机器学习模型时的数据集。接着介绍了分类算法的选择过程，给出了应用于缓存系统时分类器应该满足的两个标准，即不错的分类准确度和较低的复杂度。然后阐述了构建模型时所用到的图片特征，以及特征选择方法。最后为确保分类结果的可信程度，尤其是分类成一次访问文件时的可信度，介绍了三种保障机制以降低误分类对缓存系统的负面影响。

5 系统测试与分析

本章对一次访问过滤的缓存系统进行测试，将介绍以下几部分内容：测试环境设计，测试数据构建、采样；分类系统的分类性能测试；一次访问过滤缓存系统在使用 LRU、FIFO、S3LRU、ARC、LIRS 等缓存替换算法时的命中率，对 SSD 的写入量，响应时间等性能测试和分类系统分解实验。

5.1 测试环境

本文的测试主要包括三个部分，即分类系统预测准确性的测试、缓存系统的性能测试、及分类系统的性能分解测试。为达到较高的性能水平，本文主要采用 C++ 语言来仿真图 3-3 所示的缓存架构，仅仅在分类器的训练过程使用了 sklearn，分类预测过算法仍使用 C++ 实现，本文的测试环境如表 5-1 所示。

表 5-1 缓存系统的测试环境

CPU	Intel(R) Xeon(R) CPU E5-2609 @ 2.50GHz
内存	32GB
固态硬盘	4TB
网卡	100Mbps
操作系统	Linux version 4.15.0-43-generic Ubuntu 18.04.1 LTS

5.2 数据来源和采样

原始日志数据收集于 QQ 相册的某 OC (Outside Cache) 缓存服务器节点，包含了 9 天的访问记录，约 58 亿次访问，总访问数据量为 50TB。为了简化处理时，使用以下步骤随机采样日志数据。首先，提取出唯一图片 id 以构建图片对象集 L。接着，在 L 上按 100:1 的比例进行随机抽样获得对象集 L'。最后，在原始日志中保留图片 id 属于对象集 L' 的记录，并按照时间顺序构建出新的日志记录。最终的样本数据集包含大约 1400 万个数据对象。本章将在采样后的日志数据上进行各类实验，将缓存空间设定在从 2GB 到 40GB 的范围，由于采样的存在，实际的缓存空间大约为 200GB 到 4TB。

5.3 分类系统的性能

本文在实验中选择以下缓存算法：LRU，ARC，S3LRU，LIRS 和 FIFO。如 3.4 节中所述，LRU，ARC，S3LRU 和 FIFO 的一次访问判定标准是相同的，而 LIRS 的标准有些不同，其中 $M_{LIRS} = M_{LRU} \times R_s$ ，其中 $R_s = C_s / C$ ， C_s 表示 LIRS 算法中 STACK (S) 的空间，它保存了有较小的重用距离的块， C 表示整个缓存空间。为了保持本文研究工作的普适性，本文分别为 LRU 和 LIRS 两种算法进行了分类系统的性能测试，其结果分别如图 5-1a 和图 5-1b 所示。

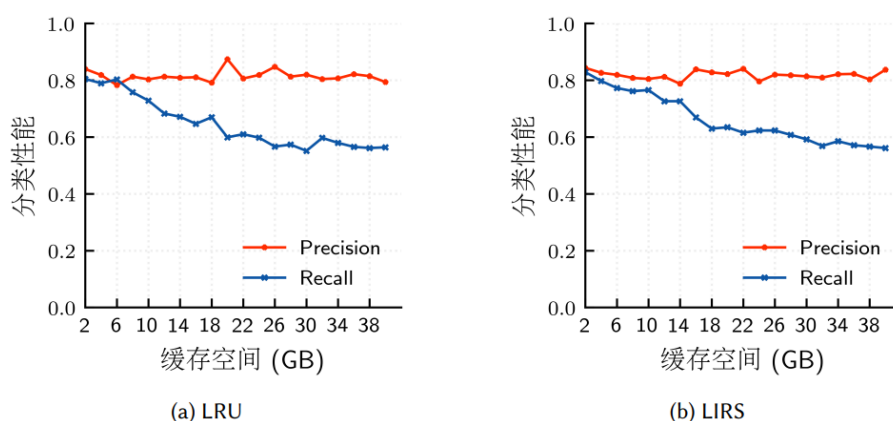


图 5-1 分类系统的性能

从图中可以看出，预测 LIRS 算法的一次访问图片准确度略高于 LRU 算法。原因在于 $M_{LIRS} < M_{LRU}$ ，这意味着对于 LRU 算法，需要预测更远的未来，其预测难度增加。但是，在 5.4 节中，我们将看到使用一次访问过滤后的 LRU 算法比 LIRS 算法命中率提升更加明显。这归因于两个因素：一是原始的 LRU 算法命中率比 LIRS 算法命中率更低，有更多的改进空间。二是高级的缓存替换算法，如 LIRS，ARC，有它们的自己的策略，以减轻一次访问文件的不利影响，从而需要更高质量分类准确度才能有进一步的命中率提升。

5.4 缓存系统的性能

本小节将从文件命中率，字节命中率，文件写入率，字节写入率，响应时间五个方面对一次访问过滤的缓存系统进行测试分析。缓存系统将在 LRU，FIFO，S3LRU，ARC，LIRS 五种不同的缓存替换下进行测试。在每组测试对比中，*Belady* 算法提供了缓存性

能的上限; *Original* 表示相应的传统缓存替换算法, 如 LRU, FIFO, S3LRU, ARC, LIRS; *Proposal* 表示在传统的缓存替换算法之上使用了一次访问过滤的缓存系统, *Ideal* 表示使用了分类准确度为 100%的理想分类器的一次访问过滤的缓存系统。

5.4.1 文件命中率

图 5-2 至图 5-6 分别给出了缓存空间的大小设定为从 2GB 到 40GB 时, LRU、FIFO、S3LRU、ARC、LIRS 五种缓存替算法在不同策略下缓存的文件命中率情况。文件命中率表示对图片的访问请求中, 有多少请求被缓存命中。其计算方法如公式(5-1)所示。

$$\text{File Hit Rate} = \text{Hit Files} / \text{Total Requests} \quad (5-1)$$

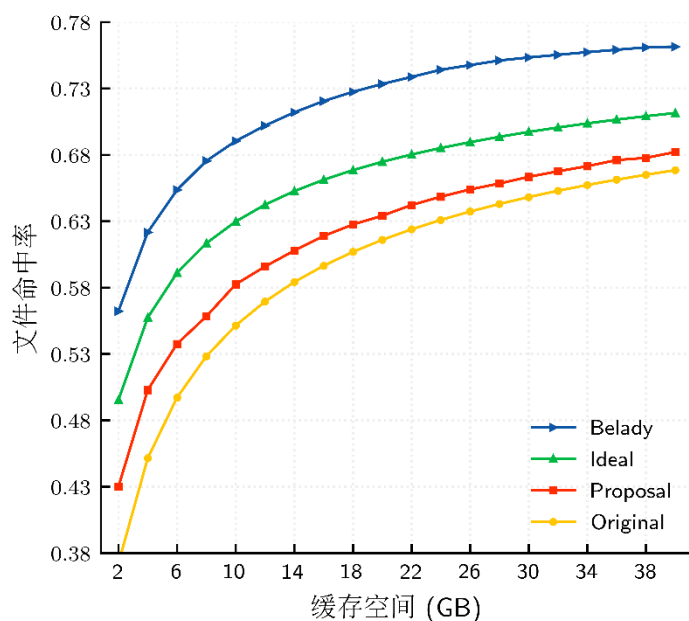


图 5-2 LRU 文件命中率

图 5-2 对比了四种不同的缓存方案时的文件命中率情况, 分别是: 原始的 LRU 缓存替换法(*Original*)、使用了本文提出的一次访问过滤策略的 LRU 替换算法(*Proposal*), 使用分类准确度为 100%的理想分类器过滤一次访问图片的 LRU 替换算法 (*Ideal*), 及理论最优的缓存替换算法 (*Belady*)。从图中可以看出, 本文提出的一次访问过滤策略较大地提升了原始 LRU 算法的缓存命中率。当缓存空间较小时, 命中率提升效果较为明显, 最大相对提升为 15.5%, 当缓存空间较大时, 提升效果较小, 最小相对提升为

1.9%。平均来看,使用了一次访问过滤策略后的 LRU 算法,命中率相对提升约为 4.4%。另外,也可以看到目前的方案与分类准确度为 100%的理想方案仍存在一定差距,拥有一定的改进空间。其它缓存替算法的命中率情况与图 5-2 类似,不再一一赘述。

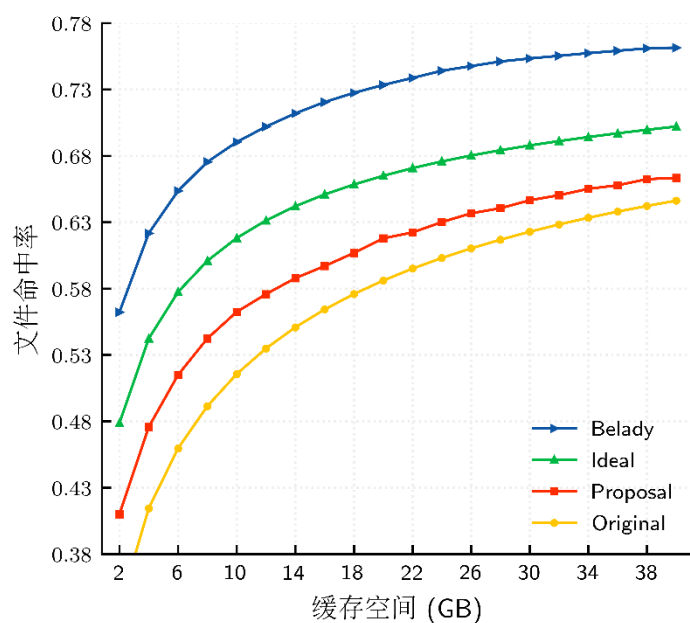


图 5-3 FIFO 文件命中率

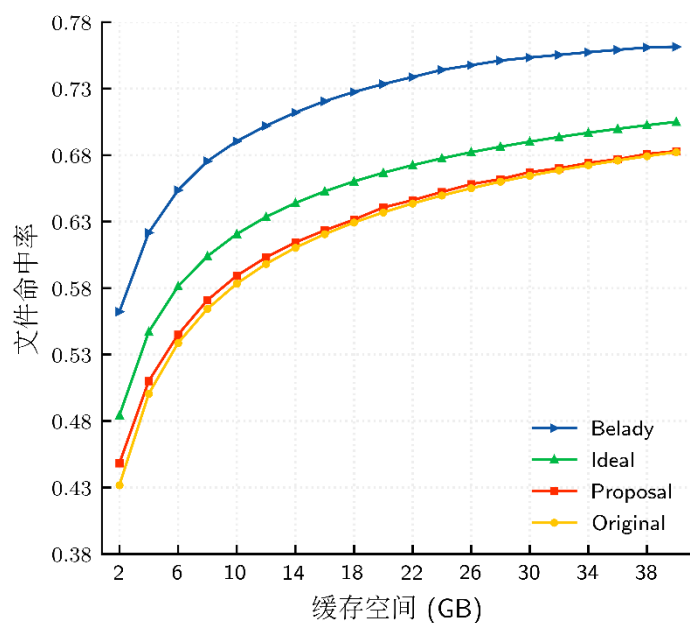


图 5-4 S3LRU 文件命中率

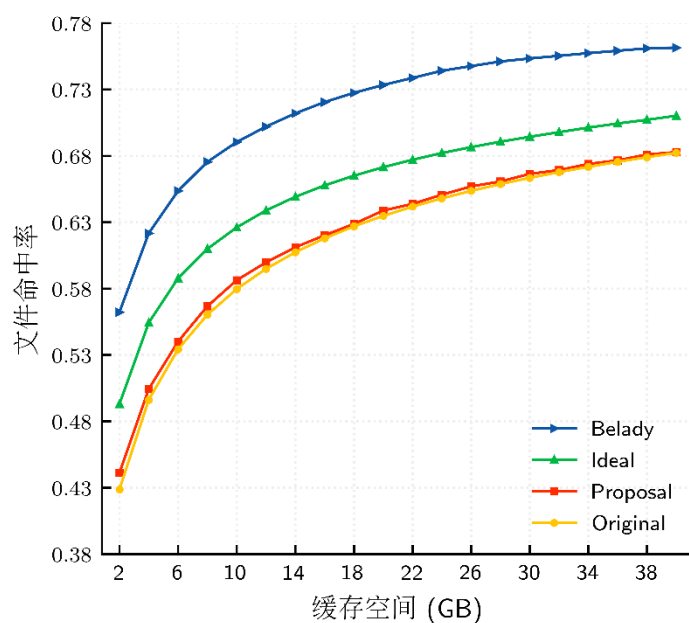


图 5-5 ARC 文件命中率

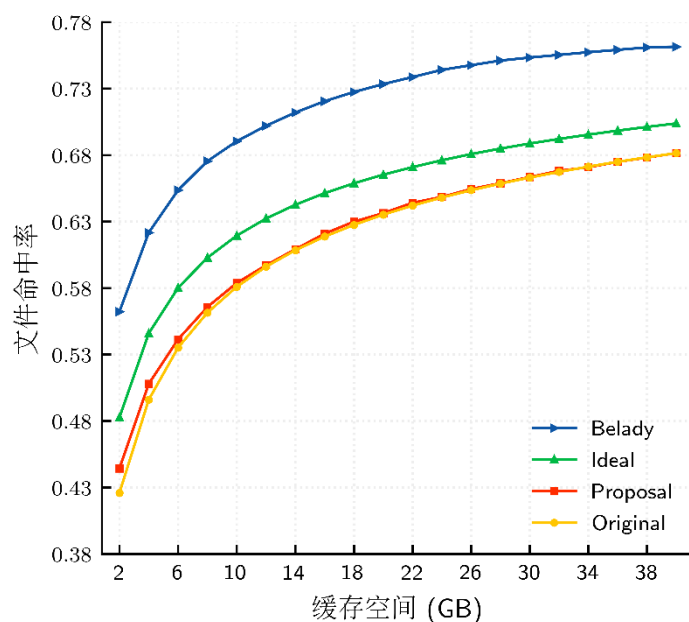


图 5-6 LIRS 文件命中率

可以从图 5-3 和图 5-2 中看出在使用分类器进行一次访问过滤之后, FIFO 和 LRU 的文件命中率提升是最高的, 它们是分别为 2.7%~20.9% 和 1.9%~15.5%。而对于高级算法, 例如 S3LRU 其命中率提升程度要小得多, 仅为 0.1%~3.8%。另外, 随着缓存容量的增加, 命中率的提升也逐渐变小。原因有两点, 一方面缓存容量增加时, 一次访问文件的判断阈值 (M) 增加, 预测难度增大, 分类器性能下降。另一方面缓存空间较

大时命中率提升空间小。这两点原因导致了一次访问策略在 S3LRU、LIRS 等高缓存替算法下，命中率的提升较小。表 5-2 更加详细地给出了五种缓存替算法的文件命中率提升情况。

表 5-2 一次访问策略对文件命中率提升

算法	最小提升	最大提升	平均提升
LRU	1.91%	15.50%	4.44%
FIFO	2.65%	20.86%	6.75%
S3LRU	0.10%	3.81%	0.74%
LIRS	0.01%	4.32%	0.56%
ARC	0.08%	2.89%	0.69%

5.4.2 字节命中率

字节命中率与文件命中率相似，所不同的是，字节命中率考虑了文件大小，表示命中数据量与整个访问数据量的比例，它表明缓存系统的吞吐量性能水平。图 5-7 至图 5-11 分别表示了 LRU、FIFO、S3LRU、ARC、LIRS 五种缓存替算算法在不同策略下的字节命中率情况，图中各个曲线的含义与图 5-2 相同。

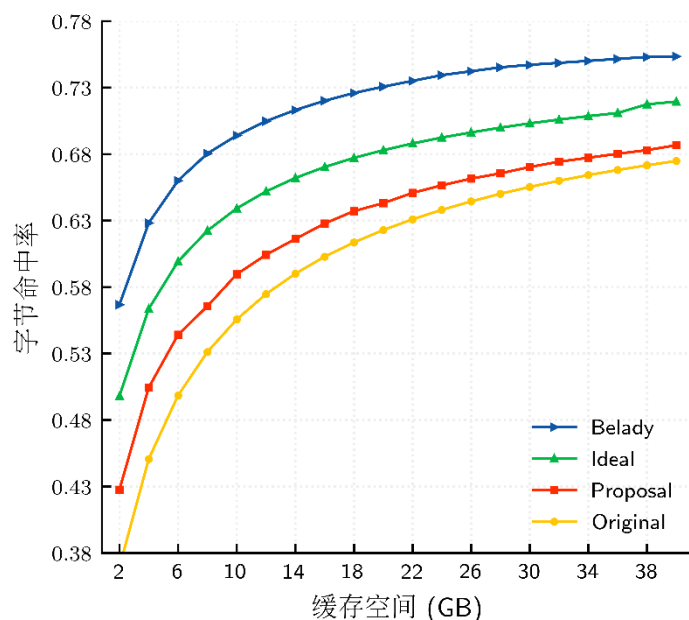


图 5-7 LRU 字节命中率

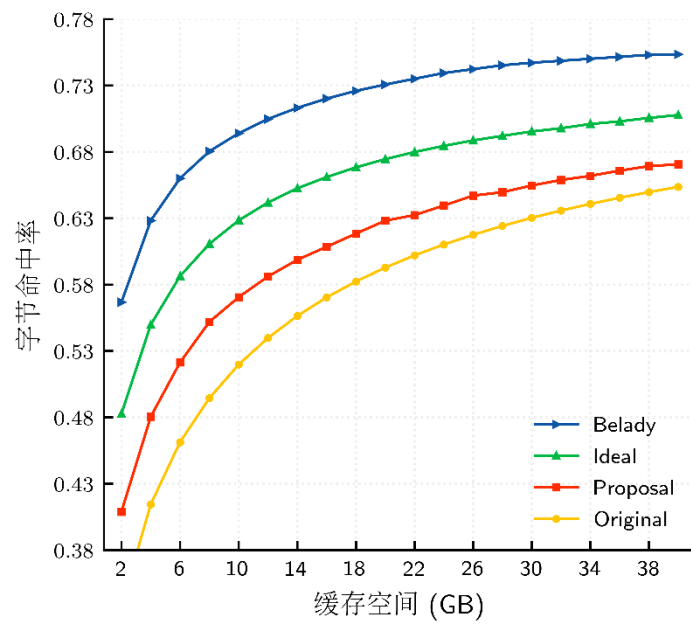


图 5-8 FIFO 字节命中率

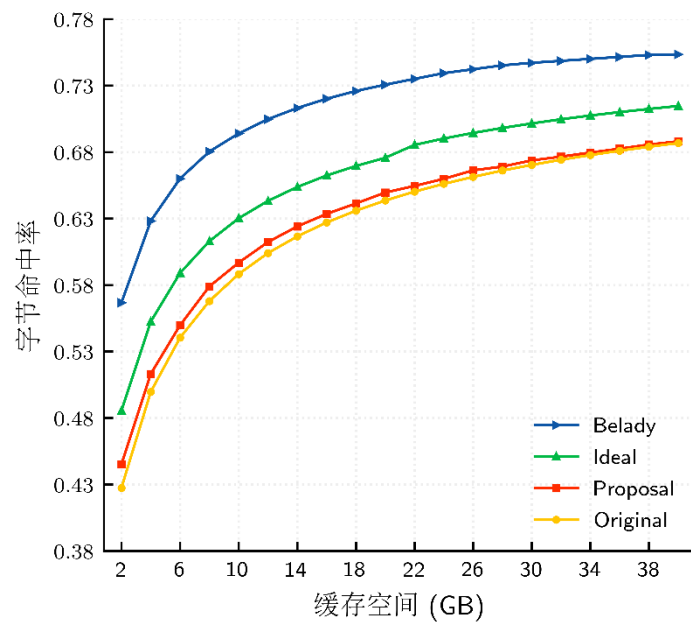


图 5-9 S3LRU 字节命中率

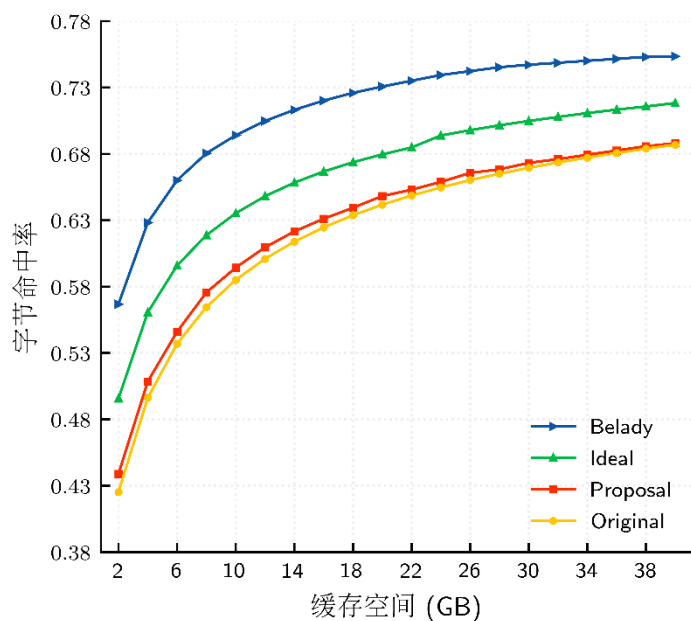


图 5-10 ARC 字节命中率

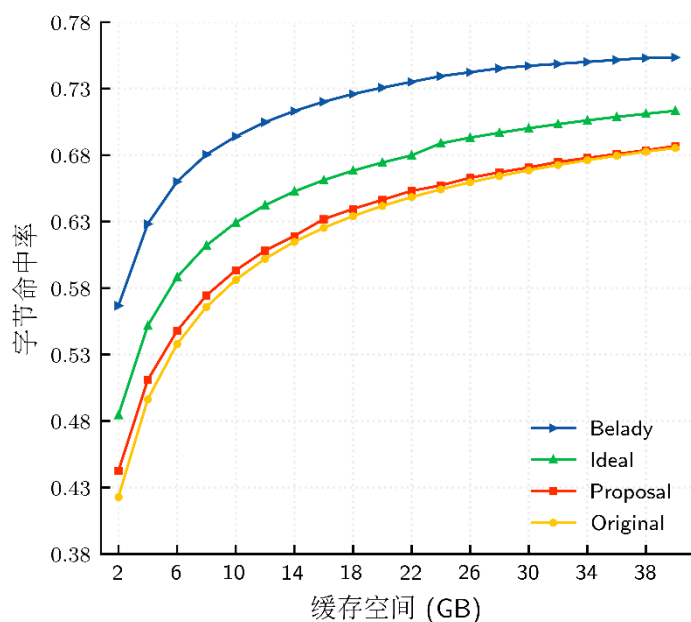


图 5-11 LIRS 字节命中率

与文件命中率类似，如图 5-8 和图 5-7 所示，使用分类器过滤一次访问图片后，FIFO 和 LRU 的提升非常明显，分别为 2.6%~21.1%和 1.7%~15.8%，但对 S3LRU 算法的命中率提升仅为 0.2%~4.1%。表 5-3 详细说明了一次访问过滤策略对五种缓存替换算法的字节命中率提升情况。由于大多数的 QQ 相册图片尺寸大致相同，并且本文设计的分类器对文件大小不敏感，因此没有观察到文件命中率和字节命中率之间存在

显著差异。

表 5-3 一次访问过滤策略对字节命中率的提升

算法	最小提升	最大提升	平均提升
LRU	1.70%	15.77%	4.65%
FIFO	2.60%	21.12%	7.24%
S3LRU	0.19%	4.14%	1.08%
LIRS	0.15%	4.65%	1.00%
ARC	0.17%	3.21%	1.06%

5.4.3 文件写入率

图 5-12 至图 5-16 分别展示了 LRU、FIFO、S3LRU、ARC、LIRS 五种缓存替换算法的 SSD 缓存文件写入率情况，它的计算方法为：写入 SSD 的文件数除以总访问文件数，如公式(5-2)所示。文件写入率表示了 SSD 缓存写入量的相比比例，能够反应 SSD 上层应用的写入情况。

$$\text{File Write Rate} = \text{Files Write to SSD Cache} / \text{Total Requests} \quad (5-2)$$

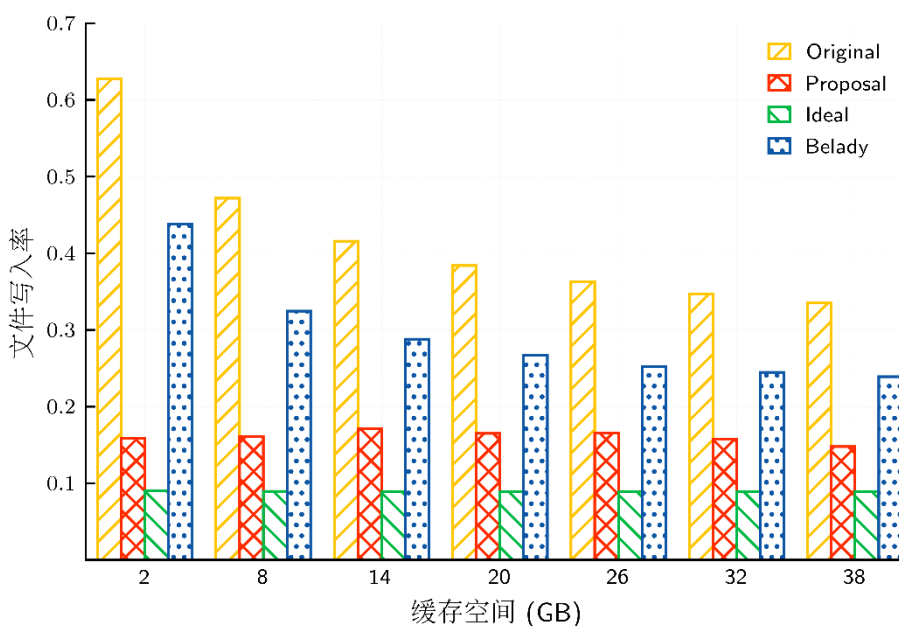


图 5-12 LRU 文件写入率

由图 5-12 可以看出，一次访问过策略极大截地减少了对 SSD 缓存的文件写入率，

相对于原始算法减少了 52.8%~77.2%。另外，使用了一次访问过滤策略的 *Proposal* 与 *Ideal*，其文件写入率皆低于传统的缓存替换算法与 *Belady*。对于传统的缓存替换算法，每次访问不命中都会导致图片从后端存储重新写入 SSD 缓存。而在一次访问过滤缓存系统中，图片访问未命中时，会首先判断图片是不是一次访问图片，只有在预测其不是一次访问图片时才会写入 SSD 缓存，从而减少了对 SSD 写入次数。

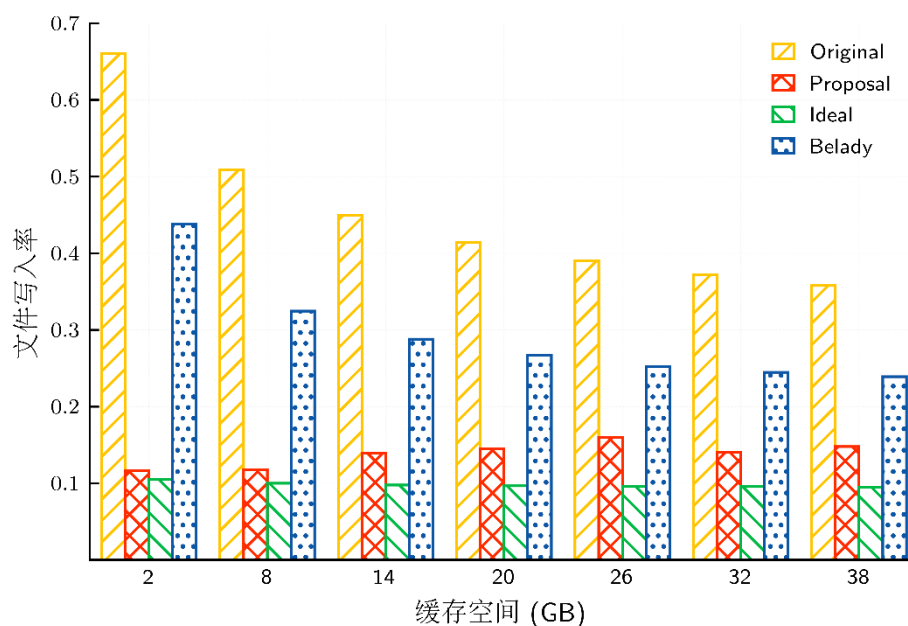


图 5-13 FIFO 文件写入率

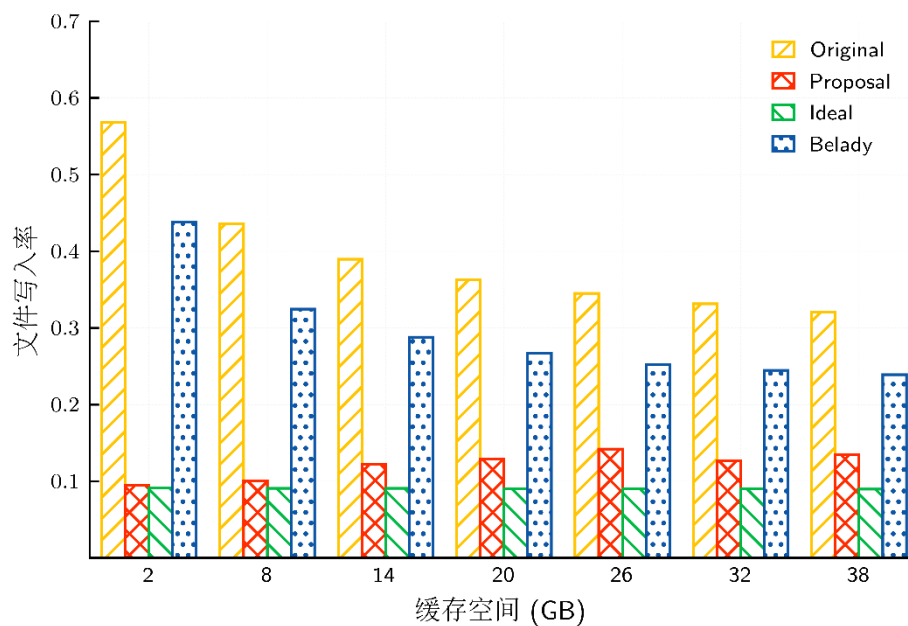


图 5-14 S3LRU 文件写入率

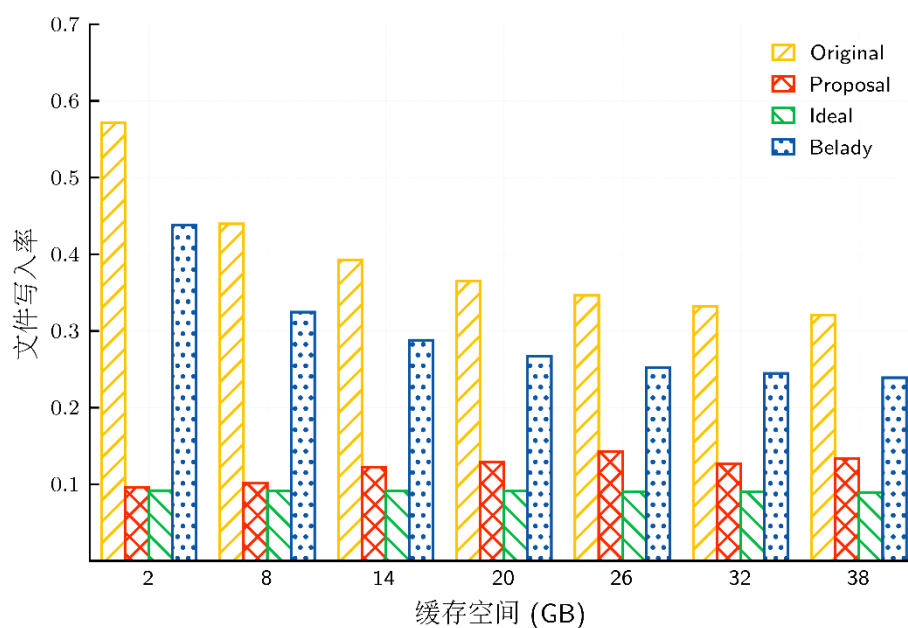


图 5-15 ARC 文件写入率

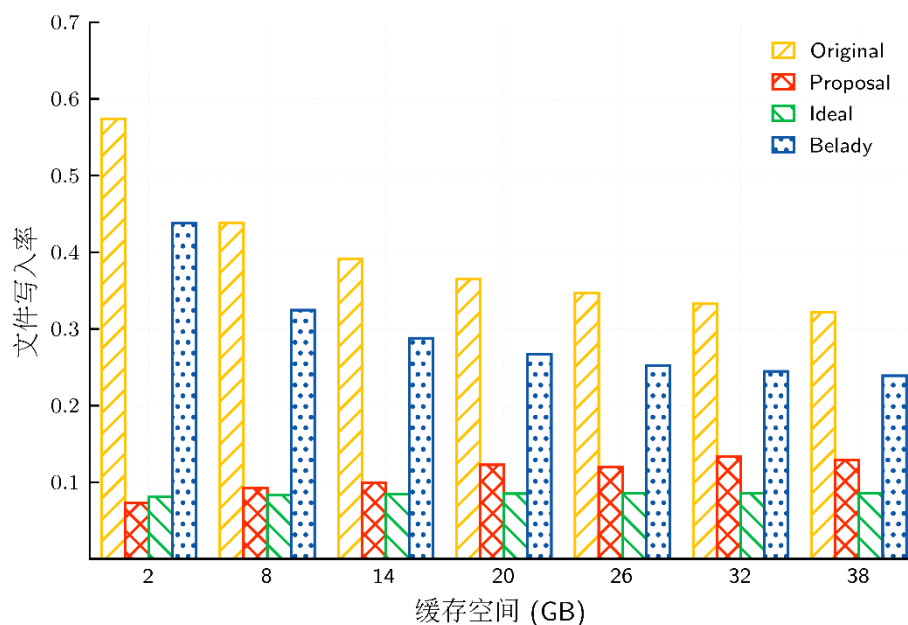


图 5-16 LIRS 文件写入率

从以上五幅图可以看出，缓存系统使用一次访问过滤策略后，五种缓存替换算法的写入率都得到了显著下降。其中，LIRS 的 SSD 文件写入率降低最为明显，相对于传统替换算法 LIRS 减少了 59.7%~87.3%。另外，也可以发现，当缓存空间较小时，一次访问策略对于减少缓存写入率的效果优越，基本与理论最优的方案 (*Ideal*) 持平。而当缓存空间较大时，为保证缓存的命中率，分类系统更偏向于将图片分类为非一次访问

图片，允许更多的图片进入缓存，故文件写入率逐渐增大。表 5-4 详细描述了一次访问过滤策略对减少文件写入率的具体效果。

表 5-4 一次访问过滤策略对文件写入率的降低

算法	最小降低量	最大降低量	平均降低量
LRU	52.76%	77.23%	60.82%
FIFO	57.99%	82.34%	66.83%
S3LRU	57.49%	83.31%	66.52%
LIRS	59.73%	87.26%	69.21%
ARC	57.66%	83.14%	66.66%

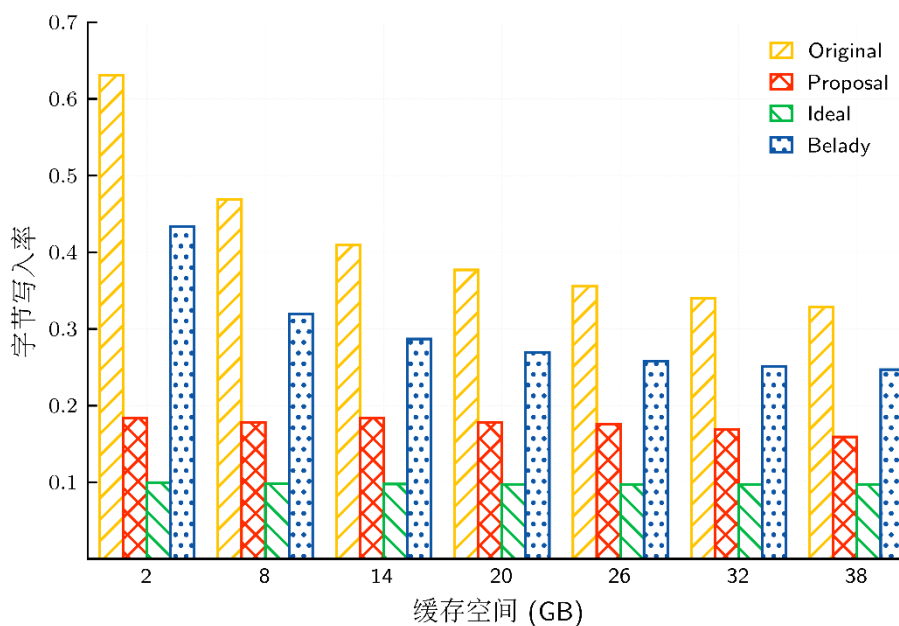


图 5-17 LRU 字节写入率

5.4.4 字节写入率

字节写入率与文件写入率的关系类似于字节命中率与文件命中率之间的关系，字节写入率考虑了文件大小，表示写入 SSD 的数据量除以总访问数据量。相对于文件写入率，字节写入率更能反应上层应用对 SSD 缓存的实际写入大小。图 5-17 至图 5-21 分别表示了 LRU、FIFO、S3LRU、ARC、LIRS 五种缓存替换算法在不同策略下的字节

写入率情况，图中各个条目的含义与图 5-2 相同。

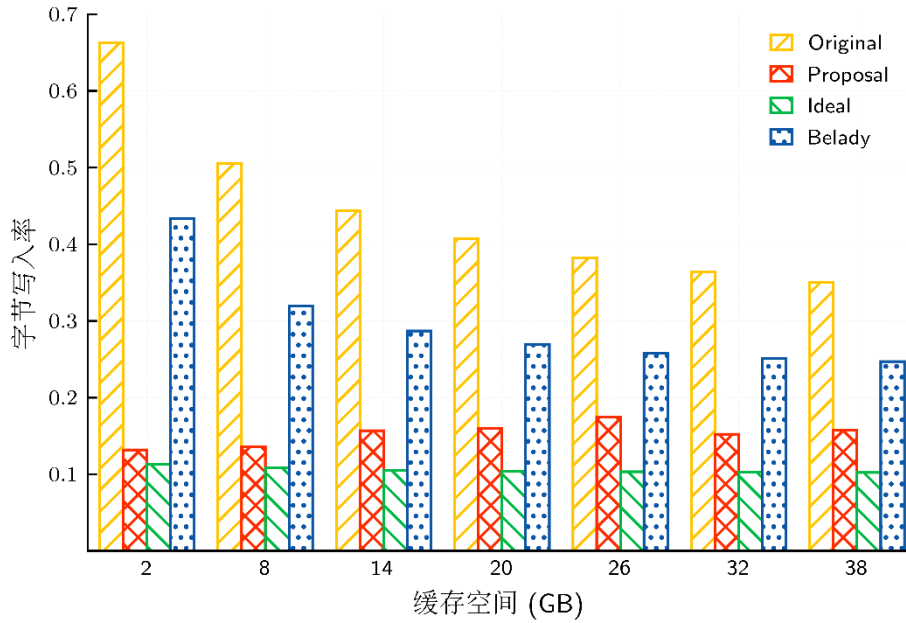


图 5-18 FIFO 字节写入率

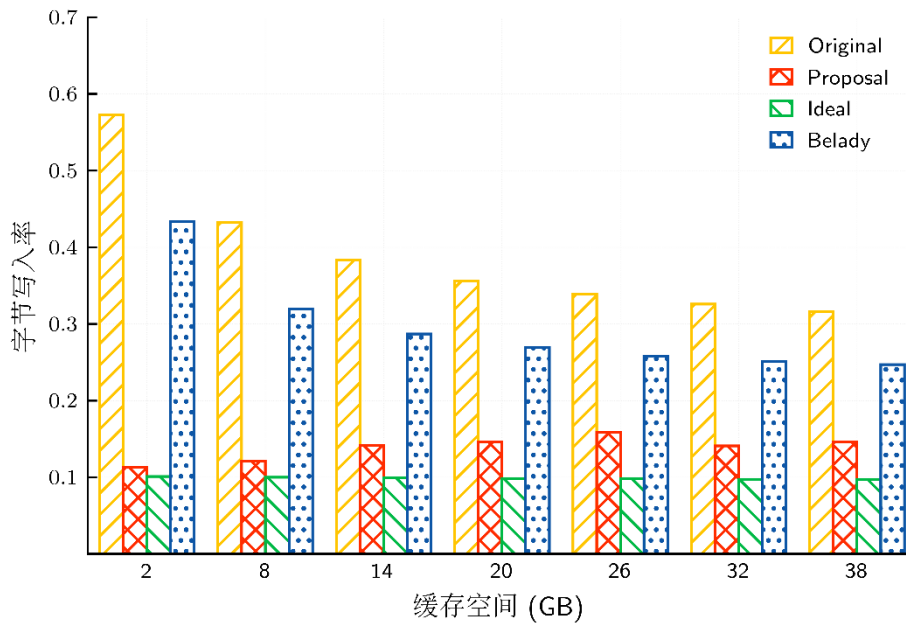


图 5-19 S3LRU 字节写入率

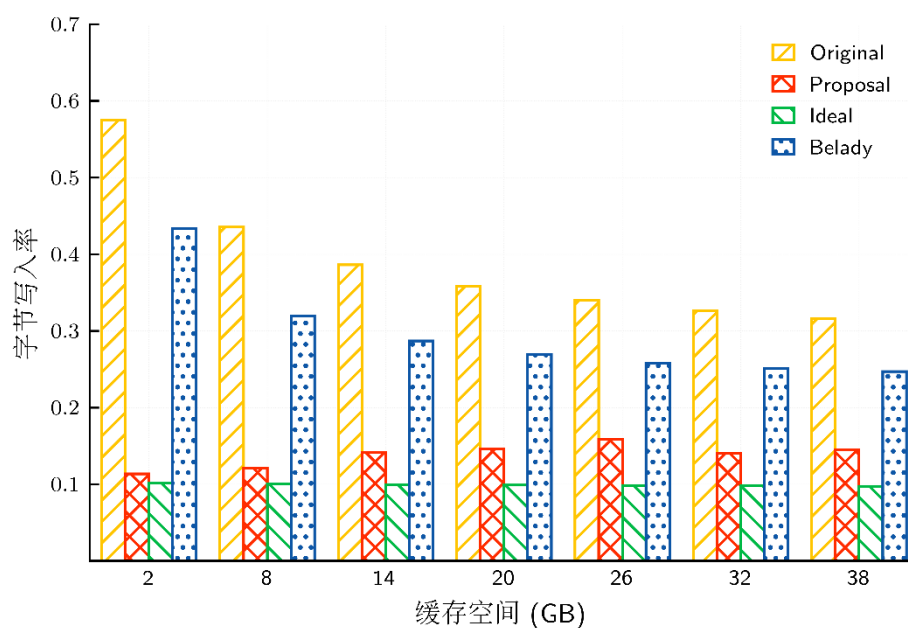


图 5-20 ARC 字节写入率

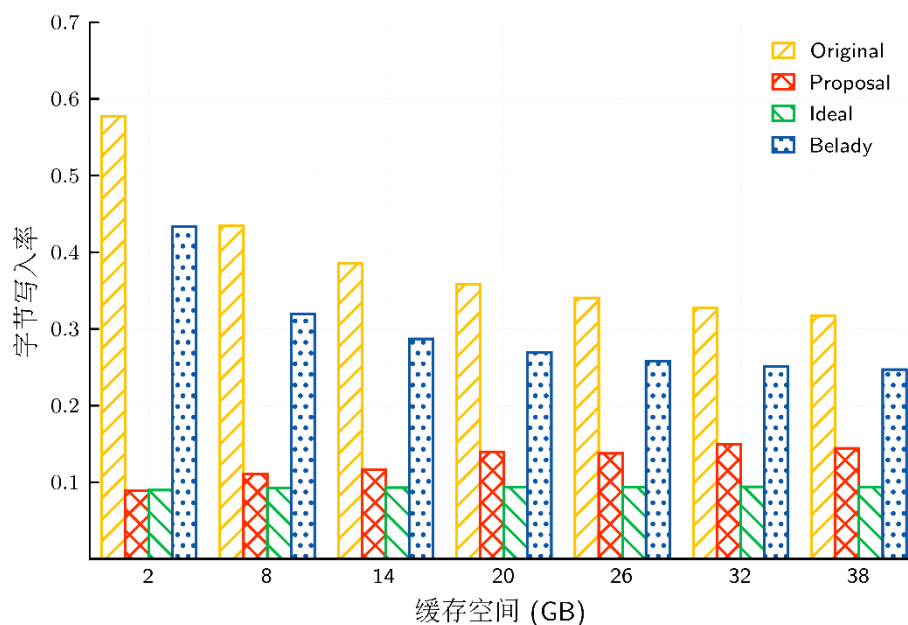


图 5-21 LIRS 字节写入率

从以上五幅图可以看出，与文件写入率类似，缓存系统在使用一次访问过滤后，五种缓存替换算法的字节写入率大大减少，其中 LIRS 减少地最多，为 54.2%~84.6%。表 5-5 给出了一次访问过滤策略对于减少 SSD 缓存字节写入率的详细效果。可以看出，在字节写入率方面，一次访问过滤策略对 FIFO、S3LRU、ARC、LIRS 四种缓存替换算

法的提升效果较为接近，均可达 80%以上。字节写入率的巨大提升表明了，本文设计的一次访问过滤策略达到了预期的目标，能够极大的减少 SSD 缓存的写入量。

表 5-5 一次访问过滤策略对字节命中率的降低

算法	最小降低量	最大降低量	平均降低量
LRU	49.13%	73.64%	56.80%
FIFO	54.14%	80.10%	62.92%
S3LRU	52.76%	80.23%	61.67%
LIRS	54.16%	84.65%	64.14%
ARC	53.01%	80.24%	61.92%

5.4.5 响应时间

由于分类系统的引入，不可避免地会对缓存系统的响应时间产生一定的影响，现就这一问题来简单讨论一下。为简单起见，用符号 t_{classify} 表示由分类器和历史表组成的分类系统进行一次访问预测的执行时间。通常，可以使用公式(5-3)来计算高速缓存系统中的平均访问时间。

$$T = \text{hit rate} \times \text{Hit cost} + (1 - \text{hit rate}) \times \text{Miss penalty} \quad (5-3)$$

无论是对于原始缓存系统还是本文提出的一次访问过滤的缓存系统，对于一个文件访问，如果文件在缓存中命中，它们的命中成本（Hit cost）是一致的。即缓存查询文件是否存在的时间（ t_{query} ）与从 SSD 读取文件时间（ t_{ssdr} ）之和，如公式(5-4)所示。

$$\text{Hit cost} = t_{\text{query}} + t_{\text{ssdr}} \quad (5-4)$$

在文件未命中的情况下，原始缓存系统的未命中代价包括缓存查询的时间和从 HDD 读取数据到 DRAM 的时间，可以用公式(5-5)表示。请注意，这里也包括将数据写入到 SSD 的过程，但不应该考虑其耗时，因为它可以在后台同时完成。

$$\text{Miss penalty}_o = t_{\text{query}} + t_{\text{hddr}} \quad (5-5)$$

其中， t_{hddr} 表示文件从硬盘读取的完成时间。

在本文提出的缓存系统中，文件未命中时首先需要对其是否是一次访问图片进行预测。如果是，它将直接从 HDD 读取到 DRAM，而不写入 SSD 缓存。如果不是一次

访问图片,它将同没有分类的原始缓存系统一样的方式进行处理,即不仅将数据从HDD读取到DRAM,同时数据会在后台被同步写入到SSD缓存。因此,本文提出的缓存系统的访问缺失代价包括缓存查询,分类预测和从HDD读取数据到DRAM。可以用公式(5-6)表示这一代价。同样,数据写入SSD的过程在后台完成,不应计算在内。

$$\text{Miss penalty}_p = t_{\text{query}} + t_{\text{classify}} + t_{\text{hddr}} \quad (5-6)$$

其中, t_{classify} 表示分类预测的执行时间。

通过这种方式,我们可以对原始缓存系统与本文提出的一次访问过滤的缓存系统二者的响应时间进行比较,如图 5-22 至图 5-26 所示。它们表示了对于一张大小 32K 图片,LRU、FIFO、S3LRU、ARC、LIRS 五种替换算法在两个缓存系统下的平均响应时间。上述公式中,设定 $t_{\text{hddr}} = 3\text{ms}$, $t_{\text{query}} = 1\mu\text{s}$, $t_{\text{classify}} = 0.4\mu\text{s}$, 请注意, t_{query} 和 t_{classify} 是实验测试的平均值。在图 5-22 中, *Original* 表示了传统缓存系统中 LRU 算法的平均响应时间, *Proposal* 表示了一次访问过滤的缓存系统中 LRU 算法的平均响应时间。可以看出一次访问过滤的缓存系统对响应时间也有一定的提升,于 LRU 算法来说,提升范围在 3.2%~9.3%之间。

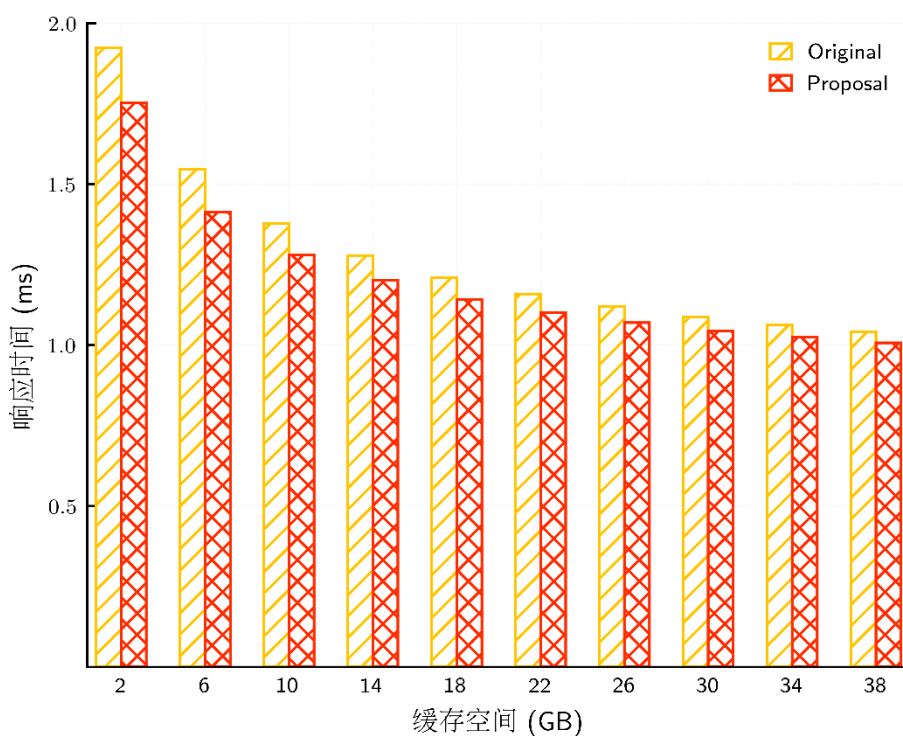


图 5-22 LRU 响应时间

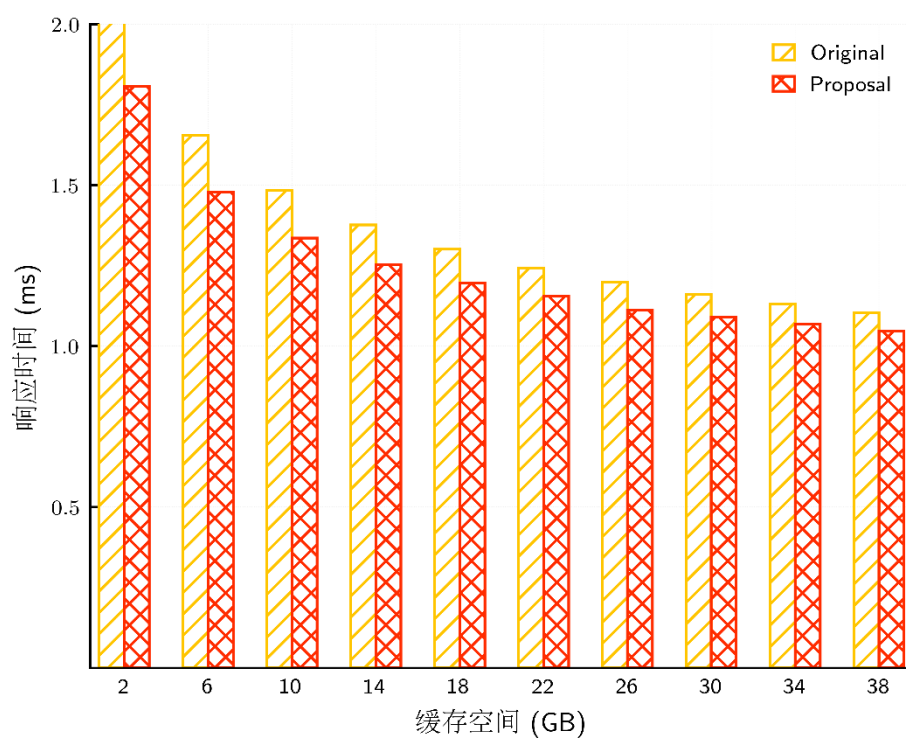


图 5-23 FIFO 响应时间

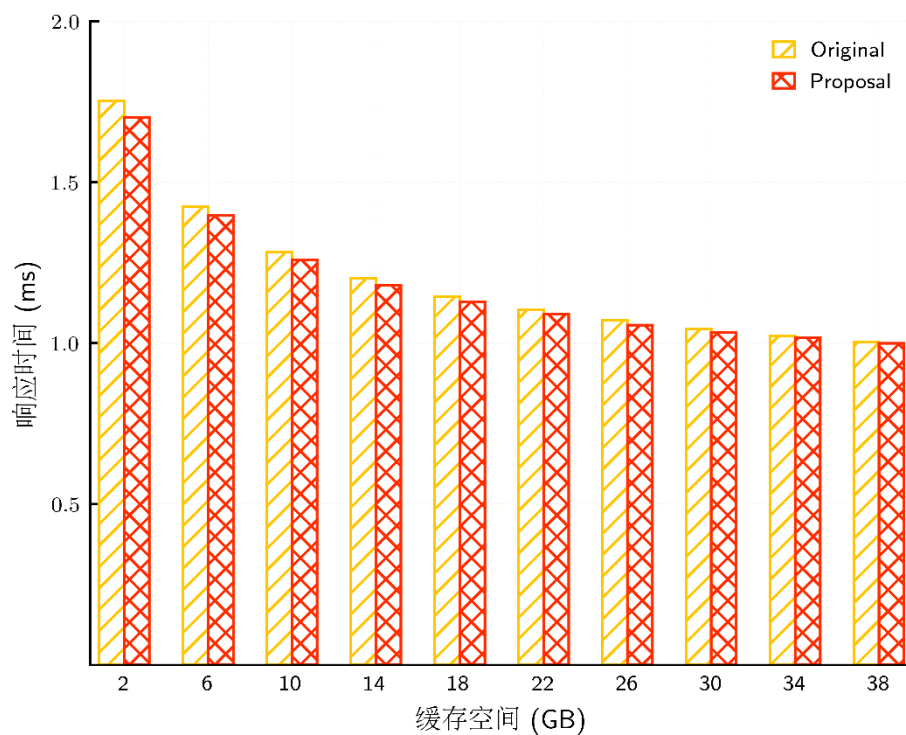


图 5-24 S3LRU 响应时间

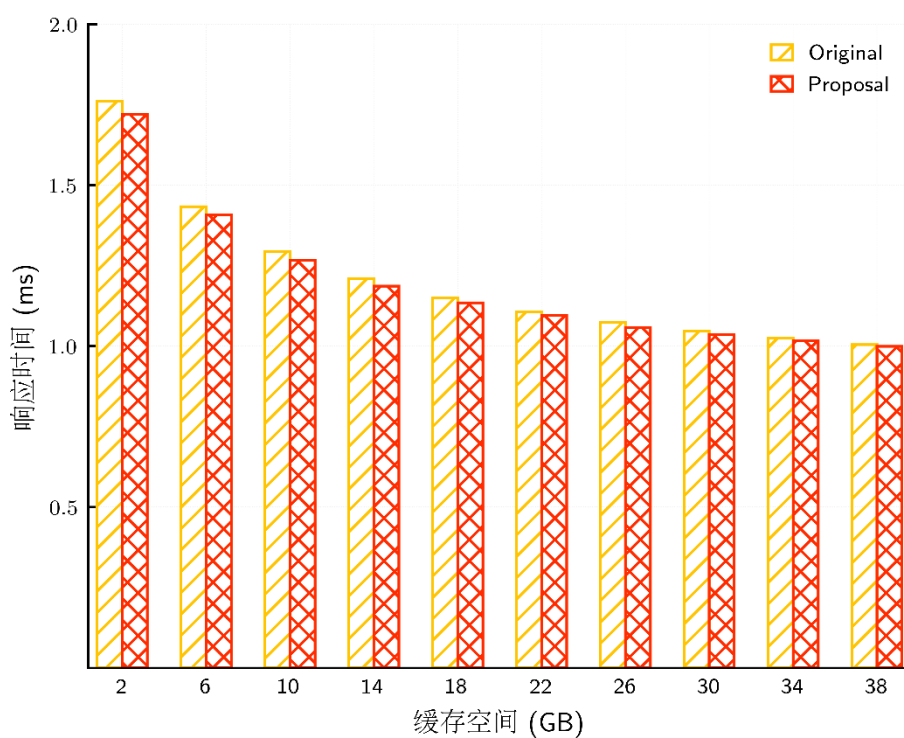


图 5-25 ARC 响应时间

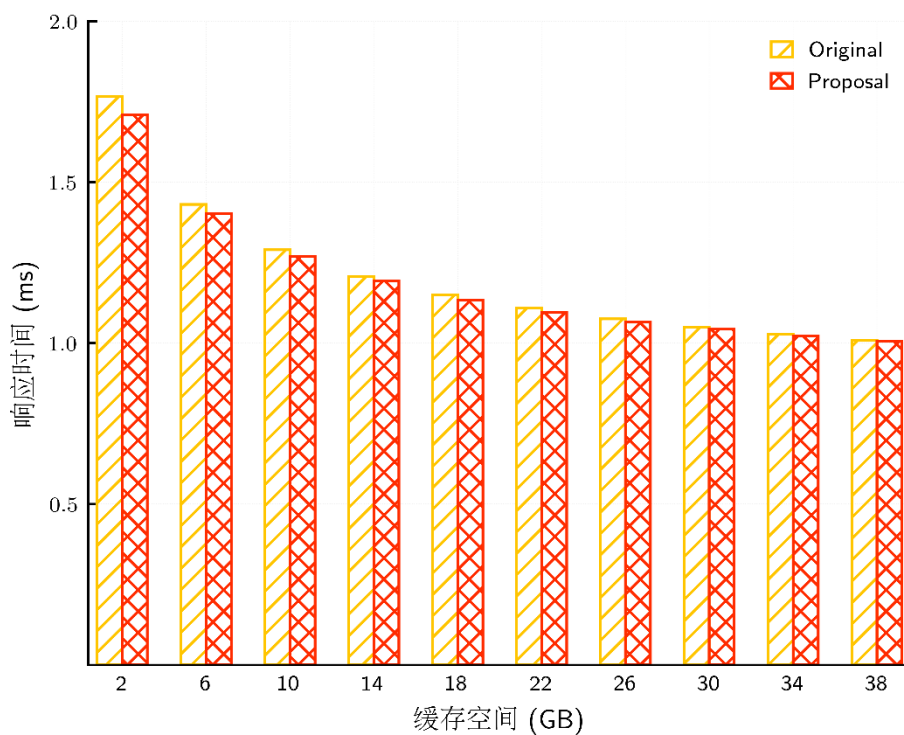


图 5-26 LIRS 响应时间

从以上五幅图可以看出，LRU 与 FIFO 算法在使用一次访问过滤策略后，缓存系统的响应时间提到了较为明显的提升，而 S3LRU、ARC 与 LIRS 算法的响应时间提升

较小。具体的提升效果数据如表 5-6 所示。由公式(5-3)至公式(5-6)可知，一次访问过滤的缓存系统，命中代价与传统系统相同，而访问缺失代价中引用入分类预测时间。因此只有当命中率提到一定的提升后才能降低用户的访问延时。由于分类预测的时间很短，仅为 1us，因此命中率只要有些许提升即可。

表 5-6 一次访问过滤策略对系统响应时间的提升

算法	最小提升	最大提升	平均提升
LRU	3.19%	9.31%	5.47%
FIFO	4.53%	10.76%	7.77%
S3LRU	0.36%	2.93%	1.38%
LIRS	0.29%	3.23%	1.22%
ARC	0.33%	2.36%	1.39%

5.5 分类系统的分解

为了进一步探讨分类器和历史表对缓存系统的影响，现将分类系统进行分解，分别测试有无历史表 and 不同分类器时一次访问过滤的缓存系统的性能表现。分类器在对非一次访问图片进行误判时，会导致对该图片后续的几次访问都不命中，历史表旨在降低这一不利影响。直观地说，如果分类器准确度越高，则历史表的作用越小。

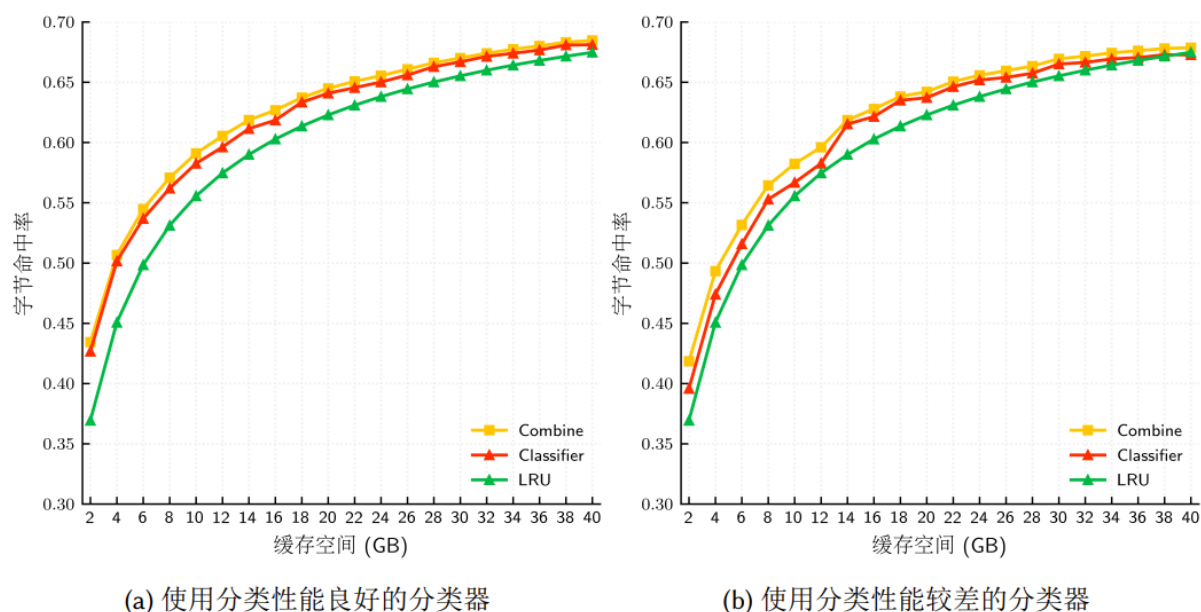


图 5-27 使用两种不同分类器时的命中率对比

为了说明历史表对不同分类器的提升效果，分别使用了两个具有不同分类性能的决策树。一个具有较高的分类准确率，约为 80%，而另一个具有较差的准确度，约为 70%。图 5-27 比较了在使用这两个分类器时的命中率。图 5-27 中的三条曲线分别代表原始 LRU 算法（LRU），只使用分类器而无历史表的 LRU 算法（Classifier），以及使用历史表和分类器的 LRU 算法（Combine）。在使用分类准确性较好的分类器时，历史表的引入使命中率提升了 0.8%（图 5-27a），在使用准确度较差的分类器时，历史表提升了 1.5% 的命中率（图 5-27b）。印证了之前的推论，即在分类器准确度越高时，历史表的作用越小。

5.6 本章小结

本章从各个维度对一次访问过滤的缓存系统进行了测试验证。首先，简要说明了测试环境的构成和测试数据的来源。接着针对一次访问的分类系统进行了测试，给出了在不同算法和空间下的分类性能。然后，对缓存系统整体进行了测试分析，从文件命中率，字节命中率，文件写入率，字节写入率，响应时间这五个方面进行评估，其中对 SSD 缓存的写入率降低十分明显，取得了不错的效果，其它方面提升效果视不同的缓存替换算法而定。最后将缓存的分类系统进行分解，验证历史表的功能。

6 全文总结与展望

当 SSD 部署到缓存层时, 写入密度变得很高, SSD 的寿命也因此受到影响。通过对真实世界的图片缓存访问日志进行分析, 本文发现可以在图片一开始进入缓存时就减少对 SSD 的写入。以 QQ 相册为例, 用户的日浏览量高达 500 亿, 其中有大量的只访问一次的图片。在传统的缓存方案中, 这些图片将被写入到缓存中, 这带来了大量对 SSD 的无效写入, 加速了 SSD 的磨损消耗, 也降低了缓存的空间利用率。

受此现象的启发, 本项研究试图预测图片未来的访问情况, 阻止只访问一次的图片进入缓存。由于云环境下海量用户访问模式的复杂性, 服务的多样性, 导致工作负载也是动态变化的, 很难使用传统的算法方案准确地完成这样的预测。因此, 本文采用机器学习方法自动地寻找最相关的特征属性, 用决策树来表达复杂的访问特性, 实现对一次访问图片的预测。针对缓存系统, 本文还设计了三种机制来保障预测结果的质量。在本文设计的方法中, 预测准确性达到了 80% 以上, 提升了缓存系统的命中率、大大减少了对 SSD 缓存的写入量, 一定程度上降低了访问延迟。虽然本文的研究对象是 QQ 相册的图片缓存系统, 但它也同样适用于其他云存储环境中的缓存。

本文的研究工作已发表在 ICPP'2018 (CCF-B) 会议中, 本项研究主要有以下贡献:

(1) 提出了一次访问过滤的缓存策略并实现了原型系统, 对 SSD 缓存的写入量减少了 54%~85%。据目前所知, 一次访问文件的判定标准是我们首次提出的。

(2) 使用机器学习方法来预测图片未来的访问情况。为了应对无历史信息预测的挑战, 提取了图片间共有的多维特征属性, 借助决策树算法使预测准确度提高到了 80% 以上。

(3) 实现了用于一次访问过滤的缓存分类器。并设计了三种保障机制来提高预测结果的质量。运用真实社交网络的访问日志进行测试, 结果表明一次访问过滤策略能有效提升 SSD 缓存多方面的性能, 包括命中率, 对 SSD 的写入量和响应时间。

对未来的展望主要包括, 考虑进一步预测图片未来的访问次数, 并结合这一预测结果设计相应的缓存替换算法。这一缓存替换算法应当结合传统的缓存替换策略与机器学习的预测结果, 并且能够容忍预测结果的错误, 在预测错误能够自适应的选择传统的替换策略淘汰数据。在本文的基础上, 进一步的提升缓存系统的命中率。

致 谢

时光荏苒，岁月如梭，转眼间三年的研究生生涯即将划上句号。回望读研这三年，感触良多，有笑有泪，相对于三年前成长了许多。在华中科技大学度过的时光是我人生中一段宝贵的经历，这里有绿意盎然的校园景色，博学多识的老师，聪明勤奋的同学，这些都深深地刻在我的脑海里。值此毕业之季，我向每一个华中大人表示衷心的感谢和美好的祝福。

首先我要感谢导师王桦副教授，她平易近人的性格，温和的处事方式，让我在整个研究生期间过得很愉快。在科研上，她凭借自身丰富的经验给了我很多指导性的意见，同时也能耐心地和我讨论一些新的观点和看法。我曾和她共同完成了几项工作，其中最为突出的是在国际会议 ICPP 上发表了一篇论文。整个过程中，王老师能够亲历亲为，为我做出科研上的示范，也引发了我对科研的兴趣。

感谢周可教授，研究生期间，他给了我很多成长的机会，同时也会耐心地指出我存在的问题，和我一同探讨研究的方向。同时，也感谢科研团队的李春花老师，在平时也给予了我一些学习上的意见和建议。感谢武汉光电国家研究中心，为我们提供了高水平研究平台及充分的研究支持，让我们可以全心全意地完成科研任务。

感谢实验室的同学们，感谢雷成师兄、汪洋涛师兄、何铭健师兄、江天明师兄、孙思师兄、刘渝师兄、王兆春学姐、李晓翠学姐、刘莉学姐，是你们带领我融入实验室这个有爱的大家庭；感谢同级的胡家琦、阴智辉、谢伟睿、戴杰、刘毅斐、张煜、邓虹雨、林立亚、杨玉娟，你们带给了我很多欢乐的时光，有你们的陪伴感觉很温馨；感谢万琨师弟、杨扬师弟、何锦彪师弟、石星师弟、胡江峰师弟、冉忒玮师妹、郭婵师妹、邢小云师妹，你们的阳光朝气给了我积极向上的动力。最后特别感谢同组的张嘉伟师弟，和我一同完成了很多科研任务，求知欲强，工作能力不俗，希望也能带领后来的师弟师妹们享受科研的乐趣。

感谢我的父母，是你们的理解与支持，让我能够站在更大的舞台，做自己想做的事，实现自己的人生理想，谢谢你们一直以来默默的付出。

感谢华中科技大学，你培养许多优秀的人才，能在这样一所高水平院校就读我感到骄傲。在往后的人生路上，我也会发扬华中大学子的优良作风，定不会让你失望。

参考文献

- [1] Karedla R, Love J S, Wherry B G. Caching strategies to improve disk system performance. *Computer, IEEE*, 1994, 27(3): 38–46
- [2] Jiang S, Zhang X. LIRS: an efficient low inter-reference recency set replacement policy to improve buffer cache performance. *ACM SIGMETRICS Performance Evaluation Review, ACM*, 2002, 30(1): 31–42
- [3] Megiddo N, Modha D S. ARC: A Self-tuning, Low Overhead Replacement Cache. in: *Proceedings of the 2Nd USENIX Conference on File and Storage Technologies*. USENIX Association, 2003(2003): 115–130
- [4] Huang P, Liu W, Tang K, et al. ROP: Alleviating Refresh Overheads via Reviving the Memory System in Frozen Cycles. in: *Parallel Processing (ICPP), 2016 45th International Conference on*. 2016: 169–178
- [5] Cidon A, Eisenman A, Alizadeh M, et al. Dynacache: Dynamic Cloud Caching. in: *7th USENIX Workshop on Hot Topics in Cloud Computing (HotCloud 15)*. USENIX Association, 2015
- [6] Yang Q, Zhang H H, Li T. Mining web logs for prediction models in WWW caching and prefetching. in: *Proceedings of the seventh ACM SIGKDD international conference on Knowledge discovery and data mining*. 2001: 473–478
- [7] Liu Z, Bai Z, Liu Z, et al. Distcache: Provable load balancing for large-scale storage systems with distributed caching. in: *17th USENIX Conference on File and Storage Technologies (FAST 19)*. 2019: 143–157
- [8] Peled L, Mannor S, Weiser U, et al. Semantic Locality and Context-based Prefetching Using Reinforcement Learning. in: *Proceedings of the 42nd Annual International Symposium on Computer Architecture*. ACM, 2015: 285–297
- [9] Keramidas G, Petoumenos P, Kaxiras S. Cache replacement based on reuse-distance prediction. in: *2007 25th International Conference on Computer Design*. 2007: 245–250

- [10] Kharbutli M, Solihin Y. Counter-Based Cache Replacement and Bypassing Algorithms. IEEE Transactions on Computers, IEEE, 2008, 57(4): 433–447
- [11] Jiménez D A, Teran E. Multiperspective reuse prediction. in: Proceedings of the 50th Annual IEEE/ACM International Symposium on Microarchitecture. 2017: 436–448
- [12] Teran E, Wang Z, Jiménez D A. Perceptron Learning for Reuse Prediction. in: The 49th Annual IEEE/ACM International Symposium on Microarchitecture. IEEE, 2016: 1–12
- [13] Qureshi M K, Srinivasan V, Rivers J a. Scalable high performance main memory system using phase-change memory technology. in: Proceedings of the 36th Annual International Symposium on Computer Architecture. ACM, 2009: 24--33
- [14] Beaver D, Kumar S, Li H C, et al. Finding a Needle in Haystack: Facebook’s Photo Storage. in: OSDI. 2010, 10(2010): 1–8
- [15] Huang Q, Birman K, Van Renesse R, et al. An analysis of Facebook photo caching. in: Proceedings of the Twenty-Fourth ACM Symposium on Operating Systems Principles. 2013: 167–181
- [16] Muralidhar S, Lloyd W, Roy S, et al. f4: Facebook’s warm blob storage system. in: Proceedings of the 11th USENIX conference on Operating Systems Design and Implementation. 2014: 383–398
- [17] Bai X, Cambazoglu B B, Russell A. Improved Caching Techniques for Large-Scale Image Hosting Services. in: Proceedings of the 39th International ACM SIGIR conference on Research and Development in Information Retrieval - SIGIR ’16. 2016: 639–648
- [18] Zhou K, Sun S, Wang H, et al. Demystifying Cache Policies for Photo Stores at Scale: A Tencent Case Study. in: Proceedings of the 2018 International Conference on Supercomputing. 2018: 284–294
- [19] Kgil T, Roberts D, Mudge T. Improving NAND flash based disk caches. in: ACM SIGARCH Computer Architecture News. 2008, 36(3): 327–338
- [20] Gray J, Fitzgerald B. Flash disk opportunity for server applications. ACM Queue, 2008,

6(4): 18–23

- [21] Tang L, Huang Q, Lloyd W, et al. RIPQ: Advanced Photo Caching on Flash for Facebook. in: Proceedings of the 13th USENIX Conference on File and Storage Technologies. USENIX Association, 2015: 373–386
- [22] Agrawal N, Prabhakaran V, Wobber T, et al. Design Tradeoffs for SSD Performance. in: USENIX Annual Technical Conference. 2008, 8: 57–70
- [23] Chang L-P, Liu Y-S, Lin W-H. Stable Greedy: Adaptive Garbage Collection for Durable Page-Mapping Multichannel SSDs. ACM Trans. Embed. Comput. Syst., New York, NY, USA: ACM, 2016, 15(1): 13:1--13:25
- [24] Zhou K, Hu S, Huang P, et al. LX-SSD: Enhancing the Lifespan of NAND Flash-based Memory via Recycling Invalid Pages. in: Proceedings of the 33rd International Conference on Massive Storage Systems and Technology. 2017
- [25] Chen F H, Yang M C, Chang Y H, et al. PWL: A progressive wear leveling to minimize data migration overheads for NAND flash devices. in: 2015 Design, Automation Test in Europe Conference Exhibition (DATE). 2015: 1209–1212
- [26] Jimenez X, Novo D, Ienne P. Wear unleveling: improving NAND flash lifetime by balancing page endurance. in: FAST. 2014, 14: 47–59
- [27] Wu G, He X. Delta-FTL: improving SSD lifetime via exploiting content locality. in: Proceedings of the 7th ACM european conference on Computer Systems. 2012: 253–266
- [28] Chen F, Luo T, Zhang X. CAFTL: A Content-Aware Flash Translation Layer Enhancing the Lifespan of Flash Memory based Solid State Drives. in: FAST. 2011, 11: 77–90
- [29] Huang P, Subedi P, He X, et al. FlexECC: Partially Relaxing ECC of MLC SSD for Better Cache Performance. in: 2014 USENIX Annual Technical Conference (USENIX ATC 14). USENIX Association, 2014: 489–500
- [30] Liu R-S, Yang C-L, Wu W. Optimizing NAND flash-based SSDs via retention

- relaxation. in: Proceedings of the 10th USENIX Conference on File and Storage Technologies. USENIX Association, 2012: 11
- [31] Soundararajan G, Prabhakaran V, Balakrishnan M, et al. Extending SSD Lifetimes with Disk-based Write Caches. in: Proceedings of the 8th USENIX Conference on File and Storage Technologies. USENIX Association, 2010: 101–114
- [32] Ye R, Meng W, Wan S. Extending Lifetime of SSD in Raid5 Systems through a Reliable Hierarchical Cache. in: Networking, Architecture, and Storage (NAS), 2017 International Conference on. 2017: 1–8
- [33] Wang H, Huang P, He S, et al. A novel I/O scheduler for SSD with improved performance and lifetime. in: Mass Storage Systems and Technologies (MSST), 2013 IEEE 29th Symposium on. 2013: 1–5
- [34] Chang L-P. Hybrid solid-state disks: combining heterogeneous NAND flash in large SSDs. in: Design Automation Conference, 2008. ASPDAC 2008. Asia and South Pacific. 2008: 428–433
- [35] Huang S, Wei Q, Feng D, et al. Improving flash-based disk cache with lazy adaptive replacement. ACM Transactions on Storage (TOS), ACM, 2016, 12(2): 8
- [36] Eisenman A, Cidon A, Pergament E, et al. Flashield: a key-value cache that minimizes writes to flash. arXiv preprint arXiv:1702.02588, 2017
- [37] Zhou K, Zhang Y, Huang P, et al. LEA: A Lazy Eviction Algorithm for SSD Cache in Cloud Block Storage. in: 2018 IEEE 36th International Conference on Computer Design (ICCD). 2018: 569–572
- [38] Breslau L, Cao P, Fan L, et al. Web caching and Zipf-like distributions: Evidence and implications. in: INFOCOM'99. Eighteenth Annual Joint Conference of the IEEE Computer and Communications Societies. Proceedings. IEEE. 1999, 1: 126–134
- [39] Petrovic S, Osborne M, Lavrenko V. Rt to win! predicting message propagation in twitter. in: Fifth International AAAI Conference on Weblogs and Social Media. 2011
- [40] Hong L, Dan O, Davison B D. Predicting popular messages in twitter. in: Proceedings

- of the 20th international conference companion on World wide web. 2011: 57–58
- [41] Zaman T, Fox E B, Bradlow E T, et al. A Bayesian approach for predicting the popularity of tweets. *The Annals of Applied Statistics*, Institute of Mathematical Statistics, 2014, 8(3): 1583–1611
- [42] Pinto H, Almeida J M, Gonçalves M A. Using early view patterns to predict the popularity of youtube videos. in: *Proceedings of the sixth ACM international conference on Web search and data mining*. 2013: 365–374
- [43] Shamma D A, Yew J, Kennedy L, et al. Viral actions: Predicting video view counts using synchronous sharing behaviors. in: *Fifth International AAAI Conference on Weblogs and Social Media*. 2011
- [44] Nwana A O, Avestimehr S, Chen T. A latent social approach to youtube popularity prediction. in: *2013 IEEE Global Communications Conference (GLOBECOM)*. 2013: 3138–3144
- [45] Khosla A, Das Sarma A, Hamid R. What makes an image popular? in: *Proceedings of the 23rd international conference on World wide web*. 2014: 867–876
- [46] Gelli F, Uricchio T, Bertini M, et al. Image popularity prediction in social media using sentiment and context features. in: *Proceedings of the 23rd ACM international conference on Multimedia*. 2015: 907–910
- [47] Tang L, Huang Q, Puntambekar A, et al. Popularity Prediction of Facebook Videos for Higher Quality Streaming. in: *2017 USENIX Annual Technical Conference (USENIX ATC 17)*. USENIX Association, 2017: 111–123
- [48] Crane R, Sornette D. Robust dynamic classes revealed by measuring the response function of a social system. *Proceedings of the National Academy of Sciences*, National Acad Sciences, 2008, 105(41): 15649–15653
- [49] Szabo G, Huberman B A. Predicting the popularity of online content. *Communications of the ACM*, ACM, 2010, 53(8): 80–88
- [50] Zhao Q, Erdogdu M A, He H Y, et al. Seismic: A self-exciting point process model for

- predicting tweet popularity. in: Proceedings of the 21th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining. 2015: 1513–1522
- [51] Sun X-H, Wang D. Concurrent average memory access time. Computer, IEEE, 2014, 47(5): 74–80
- [52] Wulf W A, McKee S A. Hitting the memory wall: implications of the obvious. ACM SIGARCH computer architecture news, ACM, 1995, 23(1): 20–24
- [53] Stallings W. Operating systems: internals and design principles. Boston: Prentice Hall, 2012
- [54] Hennessy J L, Patterson D A. Computer architecture: a quantitative approach. Elsevier, 2011
- [55] ARM Cortex-R series processors manual. <http://infocenter.arm.com/help/index.jsp?topic=/com.arm.doc.set.cortexr/index.html>
- [56] Chou H-T, DeWitt D J. An evaluation of buffer management strategies for relational database systems. Algorithmica, Springer, 1986, 1(1–4): 311–336
- [57] Bez R, Camerlenghi E, Modelli A, et al. Introduction to flash memory. Proceedings of the IEEE, IEEE, 2003, 91(4): 489–502
- [58] Wiki - Flash.[2019-04-09]. <http://wiki.csie.ncku.edu.tw/embedded/Flash>
- [59] Grupp L M, Caulfield A M, Coburn J, et al. Characterizing flash memory: anomalies, observations, and applications. in: 2009 42nd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO). 2009: 24–33
- [60] Samuel A L. Some studies in machine learning using the game of Checkers. IBM JOURNAL OF RESEARCH AND DEVELOPMENT, 1959: 71–105
- [61] Alpaydin E. Introduction to machine learning. MIT press, 2014
- [62] Weiss M A. Data Structures and Algorithms. Benjamin/Cummings, 1992
- [63] Breiman L, Friedman J H, Olshen R A, et al. Classification and Regression Trees. Wadsworth, 1984
- [64] Deng Z, Zhang L, Mishra N, et al. Memory cocktail therapy: a general learning-based

- framework to optimize dynamic tradeoffs in NVMs. in: Proceedings of the 50th Annual IEEE/ACM International Symposium on Microarchitecture. 2017: 232–244
- [65] Elkan C. The foundations of cost-sensitive learning. in: International joint conference on artificial intelligence. 2001, 17(1): 973–978

附录 硕士研究生期间的研究成果

- [1] Wang H, Yi X, Huang P, et al. Efficient SSD Caching by Avoiding Unnecessary Writes using Machine Learning. in: Proceedings of the 47th International Conference on Parallel Processing. ACM, 2018: 82
- [2] 王桦, 周可, 王兆春, 易锌波. 一种基于分类预测的缓存选择方法和系统. 发明专利. 专利申请号: 201710397520.4. 申请日期: 2017 年 05 月 31 日
- [3] 王桦、周可、易锌波、程彬、肖志立. 一种照片分类器训练方法、SSD 缓存系统及缓存方法. 专利申请号: 201810911523.X. 申请日期: 2018 年 08 月 12 日